

2. ESTRATEGIAS DE BÚSQUEDA NO INFORMADAS (PARTE 2)

IA 3.2 - Programación III

1º C - 2026

Dr. Mauro Lucci



Repaso

— — —

- La **optimalidad** de BFS depende de los costos individuales.
- La **completitud** de DFS depende de la finitud del árbol de búsqueda y no garantiza **optimalidad**.

¿Podemos mejorarlos?

Estrategias de búsqueda no informadas (avanzadas)

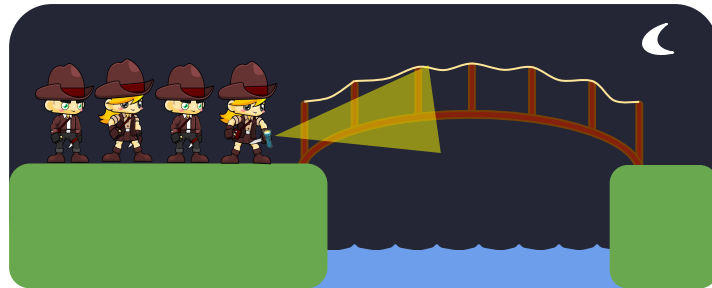
Problema de cruce de puente

— — —



Descripción

— — —



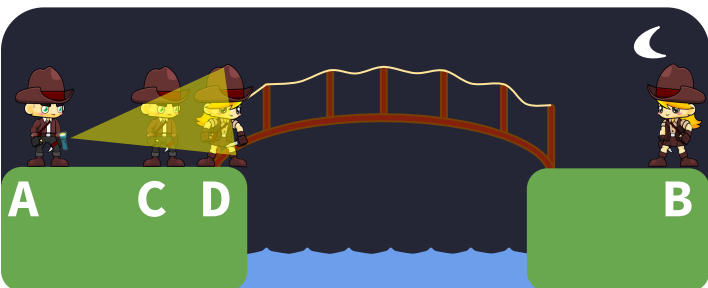
Objetivo. Las 4 personas deben llegar al otro lado del río, en el menor tiempo posible.

Reglas.

1. El río sólo se puede cruzar por el puente.
2. Por el puente pueden cruzar hasta dos personas a la vez.
3. No se puede cruzar sin la linterna.
4. La linterna puede cambiar de persona en cualquiera de los lados del río.
5. A tarda 1 segundo en cruzar el puente, B tarda 2 segundos y C y D tardan 8 segundos. Cuando dos personas cruzan por el puente, el tiempo requerido es el máximo entre los dos.

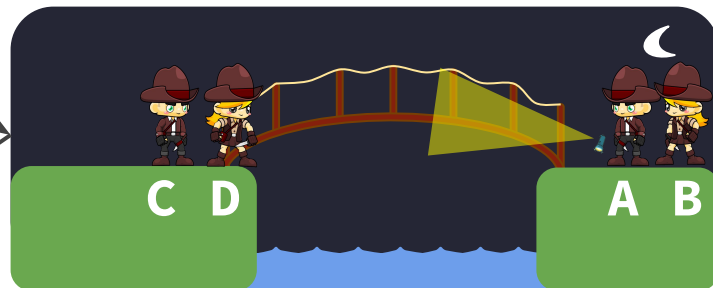


Ejemplo



Cruzan A y B
+2 s

Cruza A
+1 s





Formulación

En esta formulación, un **estado** es una 5-tupla

$$(x_A, x_B, x_C, x_D, x_L) \in \{0,1\}^5$$

que, para $i \in \{A, B, C, D, L\}$ (L es la linterna), $x_i = 1$ representa que i se encuentra en el lado izquierdo del río y $x_i = 0$, en caso contrario. El número total de estados es $2^5 = 32$.

Una **acción** es una 4-tupla

$$(y_A, y_B, y_C, y_D) \in \{0,1\}^4 \text{ tal que } 1 \leq y_A + y_B + y_C + y_D \leq 2$$

que, para $i \in \{A, B, C, D\}$, $y_i = 1$ representa que i cruza el puente e $y_i = 0$, en caso contrario.



Ejemplo



$(1, 1, 1, 1, 1)$

$(1, 1, 0, 0)$

$(1, 0, 0, 0)$



$(1, 0, 1, 1, 1)$



$(0, 0, 1, 1, 0)$



Formulación

1. Estado inicial.

$(1, 1, 1, 1, 1)$

2. Acciones. Dado un estado $(x_A, x_B, x_C, x_D, x_L)$,

$\text{ACCIONES}(x_A, x_B, x_C, x_D, x_L) =$

$$\left\{ \begin{array}{ll} \{(y_A, y_B, y_C, y_D) \in \{0, 1\}^4 : \begin{array}{l} y_i \leq x_i \text{ para todo } i \in \{A, B, C, D\}, \\ 1 \leq y_A + y_B + y_C + y_D \leq 2 \end{array} & \text{si } x_L = 1 \\ \{(y_A, y_B, y_C, y_D) \in \{0, 1\}^4 : \begin{array}{l} y_i \leq 1 - x_i \text{ para todo } i \in \{A, B, C, D\} \\ 1 \leq y_A + y_B + y_C + y_D \leq 2 \end{array} & \text{si } x_L = 0 \end{array} \right.$$

Esta desigualdad fuerza a que $y_i = 0$ cuando $x_i = 0$, es decir, no permite que i cruce al lado derecho cuando i no se encuentra en el lado izquierdo.



Formulación

3. **Modelo de transiciones.** Dado un estado $(x_A, x_B, x_C, x_D, x_L)$ y una acción (y_A, y_B, y_C, y_D) ,

$$\text{RESULTADO}((x_A, x_B, x_C, x_D, x_L), (y_A, y_B, y_C, y_D)) = \begin{cases} (x_A - y_A, x_B - y_B, x_C - y_C, x_D - y_D, 0) & \text{si } x_L = 1 \\ (x_A + y_A, x_B + y_B, x_C + y_C, x_D + y_D, 1) & \text{si } x_L = 0 \end{cases}$$

4. **Test objetivo.** Dado un estado S ,

$$\text{TEST-OBJETIVO}(S) = (S == (0, 0, 0, 0, 0))$$

5. **Costo de camino.** Es la suma de los costos individuales de cada acción del camino.

Para todo $i \in \{A, B, C, D\}$, sea t_i el tiempo que tarda i en cruzar.

Costo individual. Dado un estado S y una acción (y_A, y_B, y_C, y_D) ,

$$\text{COSTO-INDIVIDUAL}(S, (y_A, y_B, y_C, y_D)) = \max\{t_i : i \in \{A, B, C, D\}, y_i = 1\}$$

Búsqueda de costo uniforme

Uniform-Cost Search (UCS)

UCS es una **generalización** de BFS que extiende la optimalidad para problemas con **cualquier función de costo individual**.

En lugar de expandir el nodo de la frontera con la menor profundidad, se expande siempre el de **menor costo de camino**.

En particular, cuando los costos individuales son unitarios, el costo de camino coincide con la profundidad y la estrategia es **similar** a BFS.

— — —



Ejemplo - Cruce de puente

— — —

Vamos a resolver el problema de cruce de puente con el algoritmo de búsqueda de grafo y usando la estrategia de búsqueda de costo uniforme (UCS).

Los estados alcanzados los vamos a almacenar en un **diccionario**, donde las **claves** son los estados y los **valores** son costos de camino. Ya veremos por qué...



Ejemplo - Cruce de puente

— — —

$n_0:$
 $(1,1,1,1,1) \ \emptyset$

FRONTERA = $\{n_0\}$

ALCANZADOS = $\{(1,1,1,1,1): \emptyset\}$



Ejemplo - Cruce de puente

— — —

n_0 :
(1,1,1,1,1) $\emptyset$

Se debe extraer a n_0 , pues es el único nodo de la frontera.

FRONTERA = $\{n_0\}$

ALCANZADOS = $\{(1,1,1,1,1): \emptyset\}$



Ejemplo - Cruce de puente

— — —

$n_0:$
 $(1,1,1,1,1) \quad \emptyset$

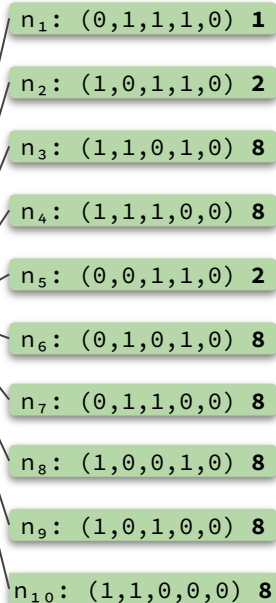
FRONTERA = $\{\}$

ALCANZADOS = $\{(1,1,1,1,1): \emptyset\}$



Ejemplo - Cruce de puente

— — —



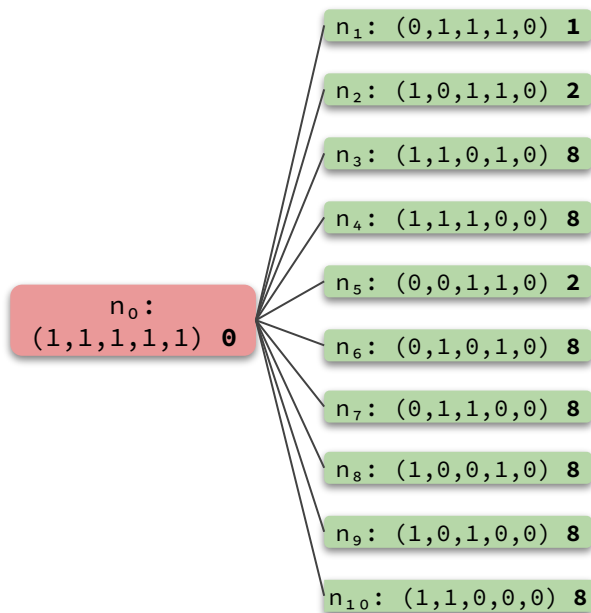
Se generaron 10 nodos, los cuales se agregan a la frontera. Además, dado que sus estados no habían sido alcanzados, se agregan al diccionario con sus respectivos costos de camino.

FRONTERA = { $n_1, n_2, n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}$ }

ALCANZADOS = {(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8}



Ejemplo - Cruce de puente



Hay 10 nodos en la frontera.
Extraemos a n_1 pues es el de
menor costo de camino.

FRONTERA = $\{n_1, n_2, n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8\}$



Ejemplo - Cruce de puente

— — —

$n_1: (0,1,1,1,0) \mathbf{1}$

$n_2: (1,0,1,1,0) \mathbf{2}$

$n_3: (1,1,0,1,0) \mathbf{8}$

$n_4: (1,1,1,0,0) \mathbf{8}$

$n_5: (0,0,1,1,0) \mathbf{2}$

$n_6: (0,1,0,1,0) \mathbf{8}$

$n_7: (0,1,1,0,0) \mathbf{8}$

$n_8: (1,0,0,1,0) \mathbf{8}$

$n_9: (1,0,1,0,0) \mathbf{8}$

$n_{10}: (1,1,0,0,0) \mathbf{8}$

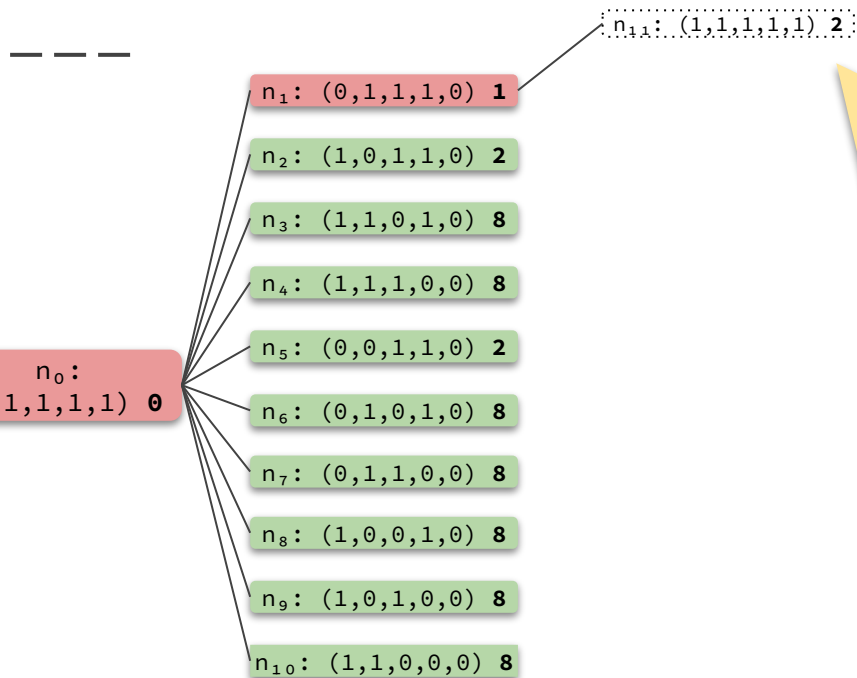
$n_0: (1,1,1,1,1) \mathbf{0}$

FRONTERA = $\{n_2, n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8\}$



Ejemplo - Cruce de puente



El nodo n_{11} contiene un estado ya **alcanzado**. Es decir, encontramos un camino redundante (en este caso particular es cíclico). Más aún, este camino **no mejora el costo de camino** que habíamos encontrado anteriormente: 0 vs. 2. Por lo tanto, **no tiene ningún sentido seguir explorando este camino** y podemos **descartar el nodo sin agregarlo a la frontera**.

Por esta razón, ALCANZADOS es un **diccionario**: para **recordar el menor costo de camino a los estados ya alcanzados**.

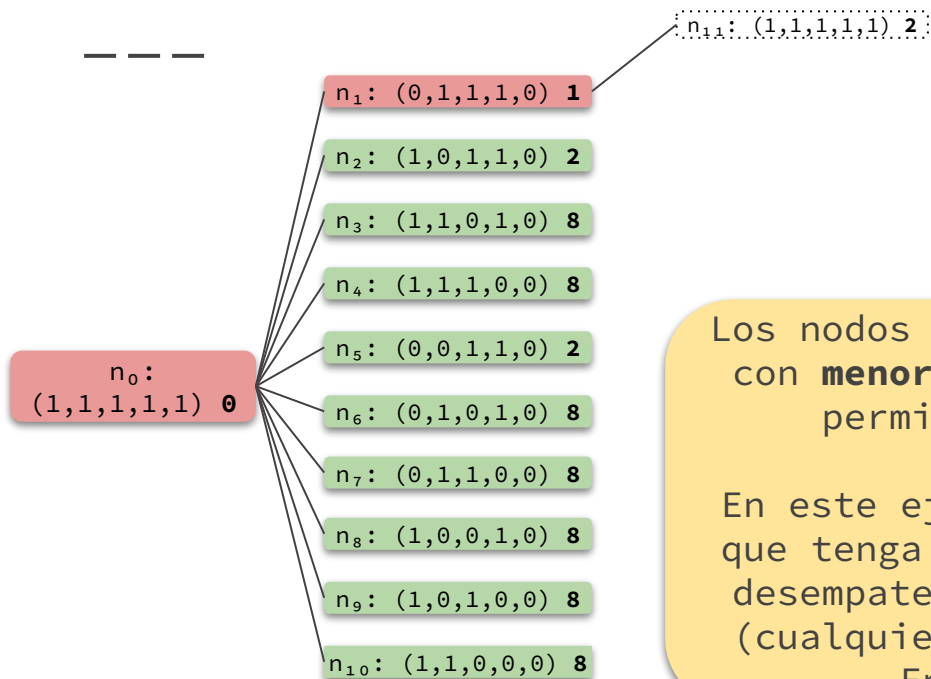
A estos nodos los vamos a dibujar con línea de puntos.

FRONTERA = $\{n_2, n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8\}$



Ejemplo - Cruce de puente



Los nodos n_2 y n_5 son los nodos de la frontera con **menor costo de camino**. La estrategia nos permite extraer **cualquiera de ellos**.

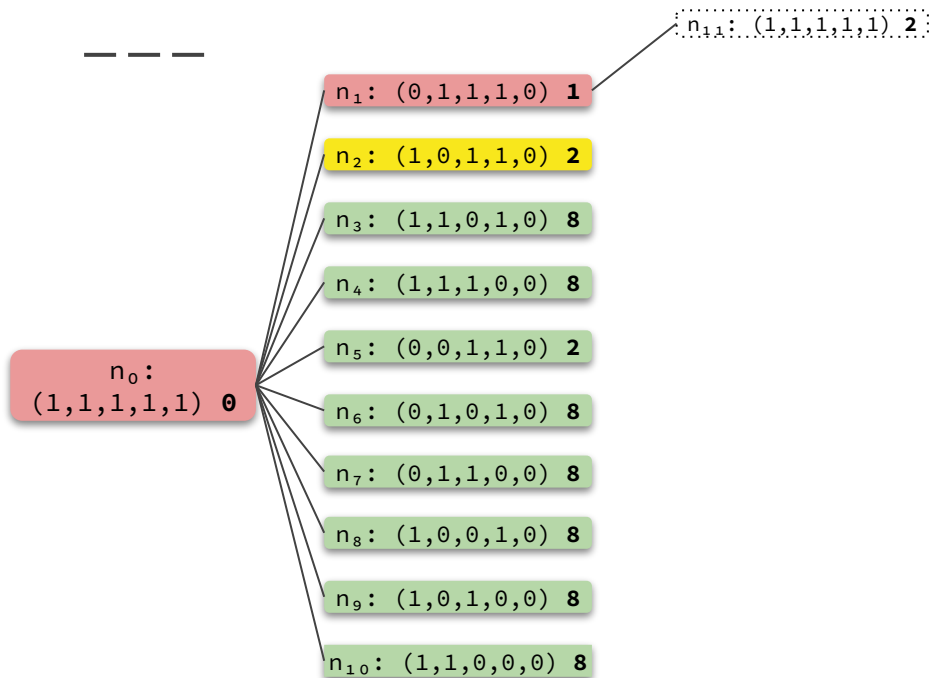
En este ejemplo, vamos a **desempatar** con aquel que tenga menor nivel y como segunda regla de desempate, su posición de arriba hacia abajo (cualquier otro criterio también es válido).
En este caso, extraemos a n_2 .

FRONTERA = $\{n_2, n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8\}$



Ejemplo - Cruce de puente

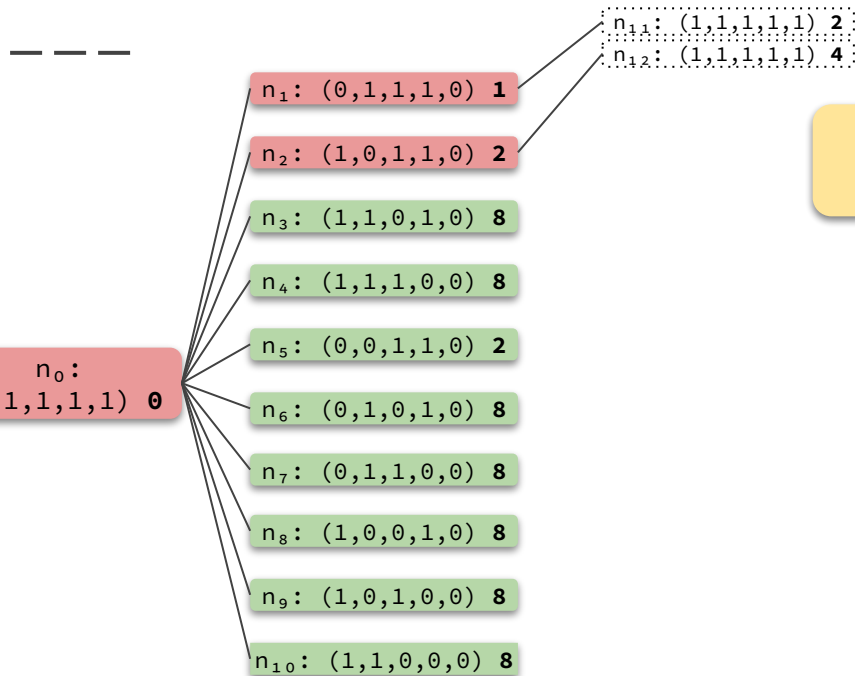


FRONTERA = $\{n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8\}$



Ejemplo - Cruce de puente



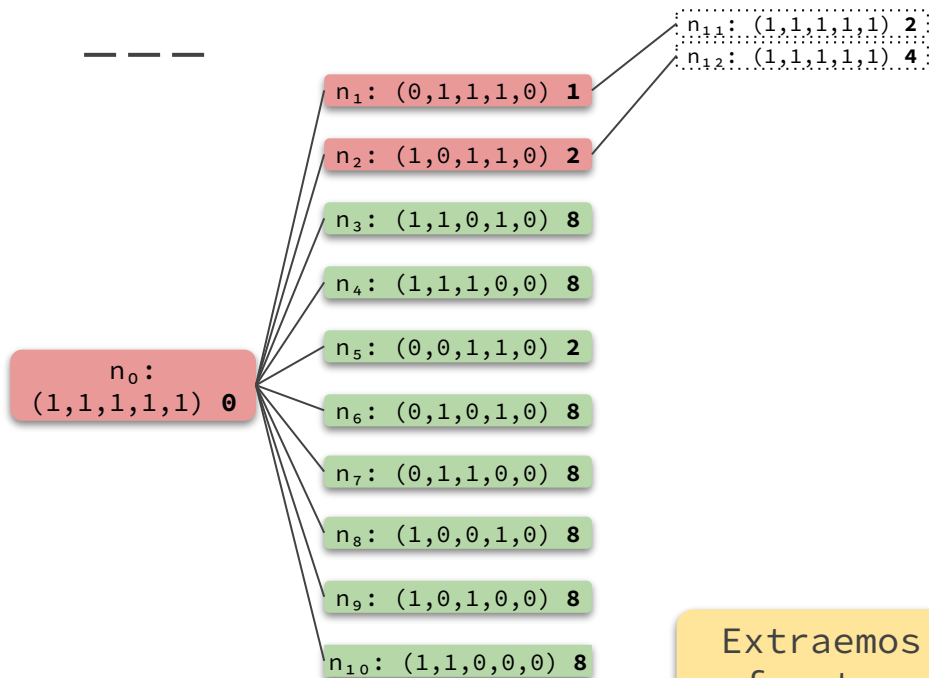
Otra vez llegamos a un estado ya alcanzado con un peor costo de camino.

FRONTERA = $\{n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8\}$



Ejemplo - Cruce de puente



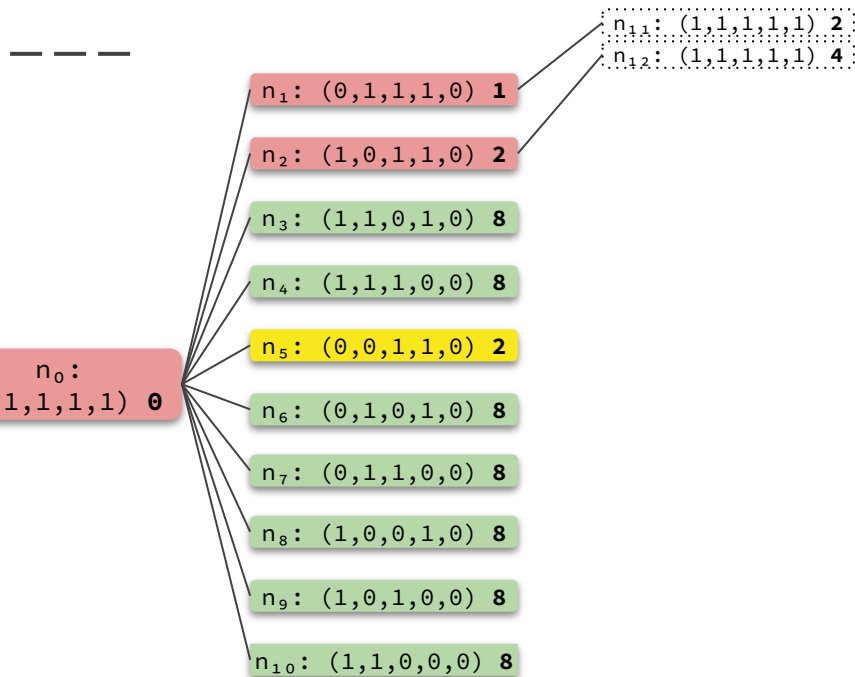
Extraemos a n_5 , pues es el nodo de la frontera con **menor costo de camino**.

FRONTERA = $\{n_3, n_4, n_5, n_6, n_7, n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8\}$



Ejemplo - Cruce de puente

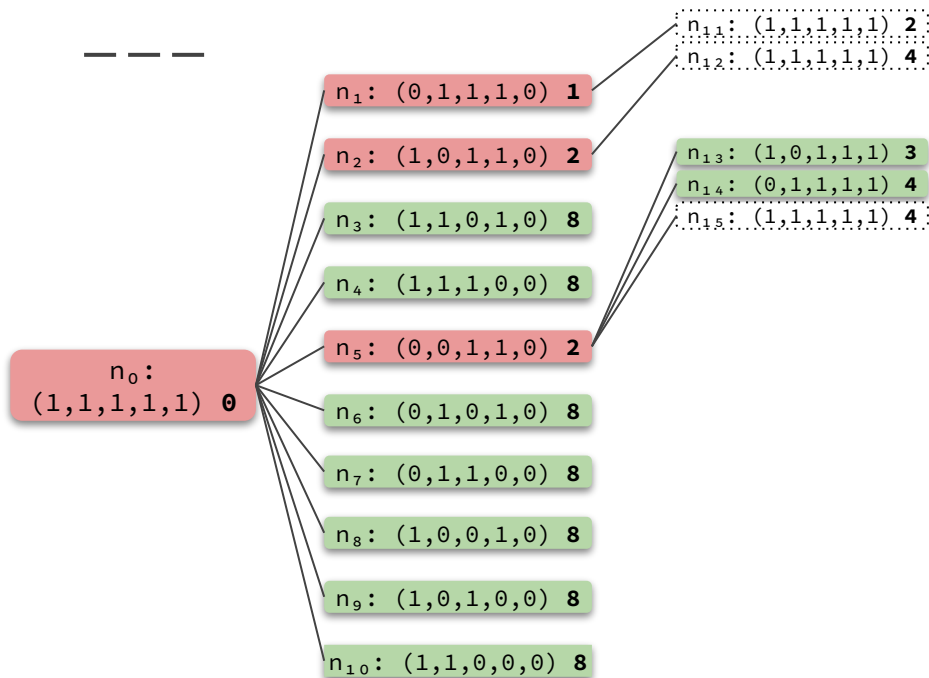


FRONTERA = $\{n_3, n_4, n_6, n_7, n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(1, 1, 1, 1, 1): 0, (0, 1, 1, 1, 0): 1, (1, 0, 1, 1, 0): 2, (1, 1, 0, 1, 0): 8, (1, 1, 1, 0, 0): 8, (0, 0, 1, 1, 0): 2, (0, 1, 0, 1, 0): 8, (0, 1, 1, 0, 0): 8, (1, 0, 0, 1, 0): 8, (1, 0, 1, 0, 0): 8, (1, 1, 0, 0, 0): 8\}$



Ejemplo - Cruce de puente



Encontramos dos estados nuevos, entonces agregamos sus nodos a la frontera y los marcamos como alcanzados. También encontramos un estado ya alcanzado con peor costo de camino, entonces lo descartamos.

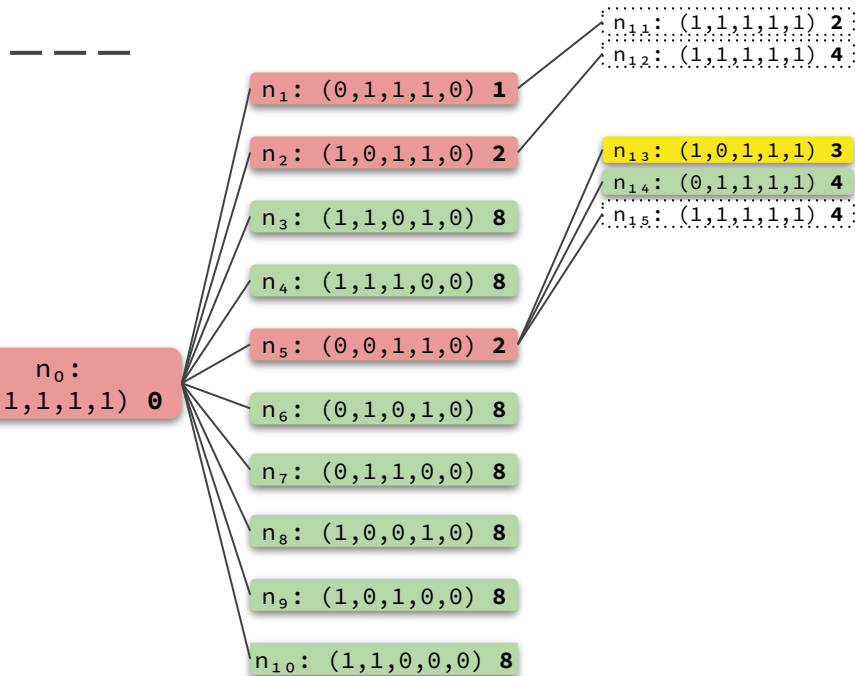
Seguimos el mismo procedimiento por algunas iteraciones más...

FRONTERA = $\{n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{13}, n_{14}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4\}$



Ejemplo - Cruce de puente

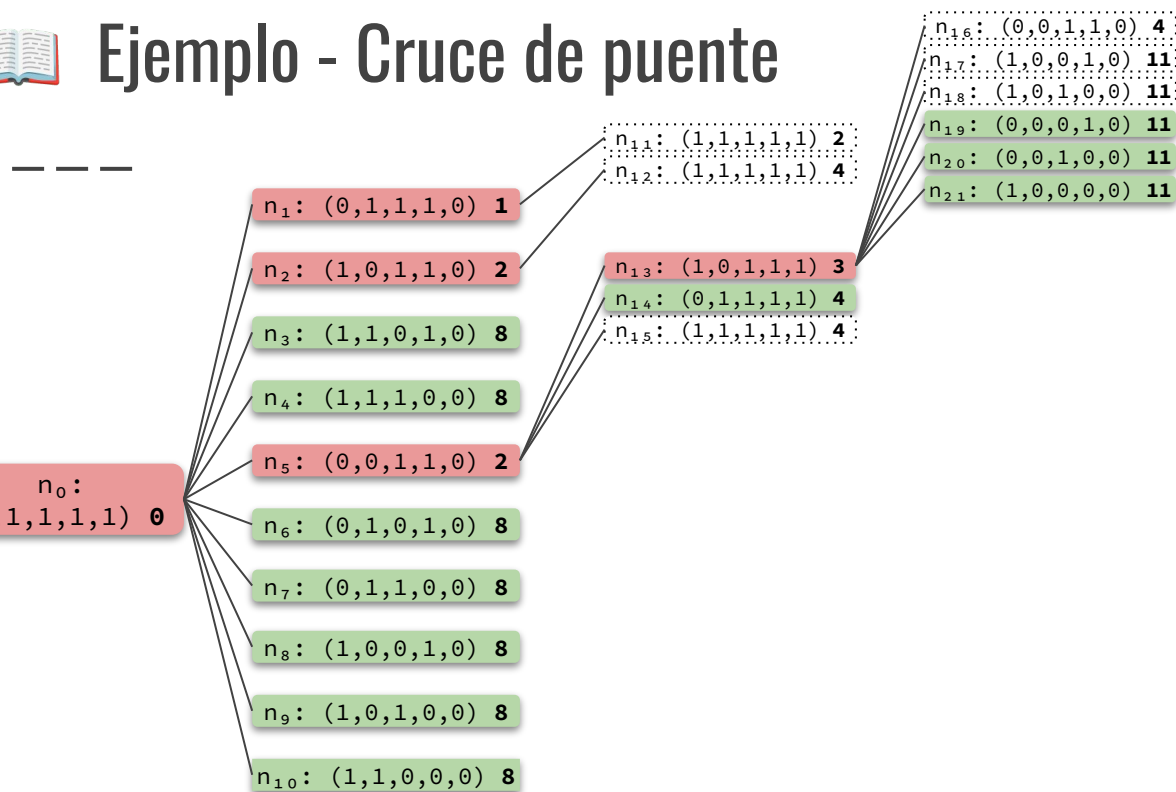


FRONTERA = $\{n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{14}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4\}$



Ejemplo - Cruce de puente



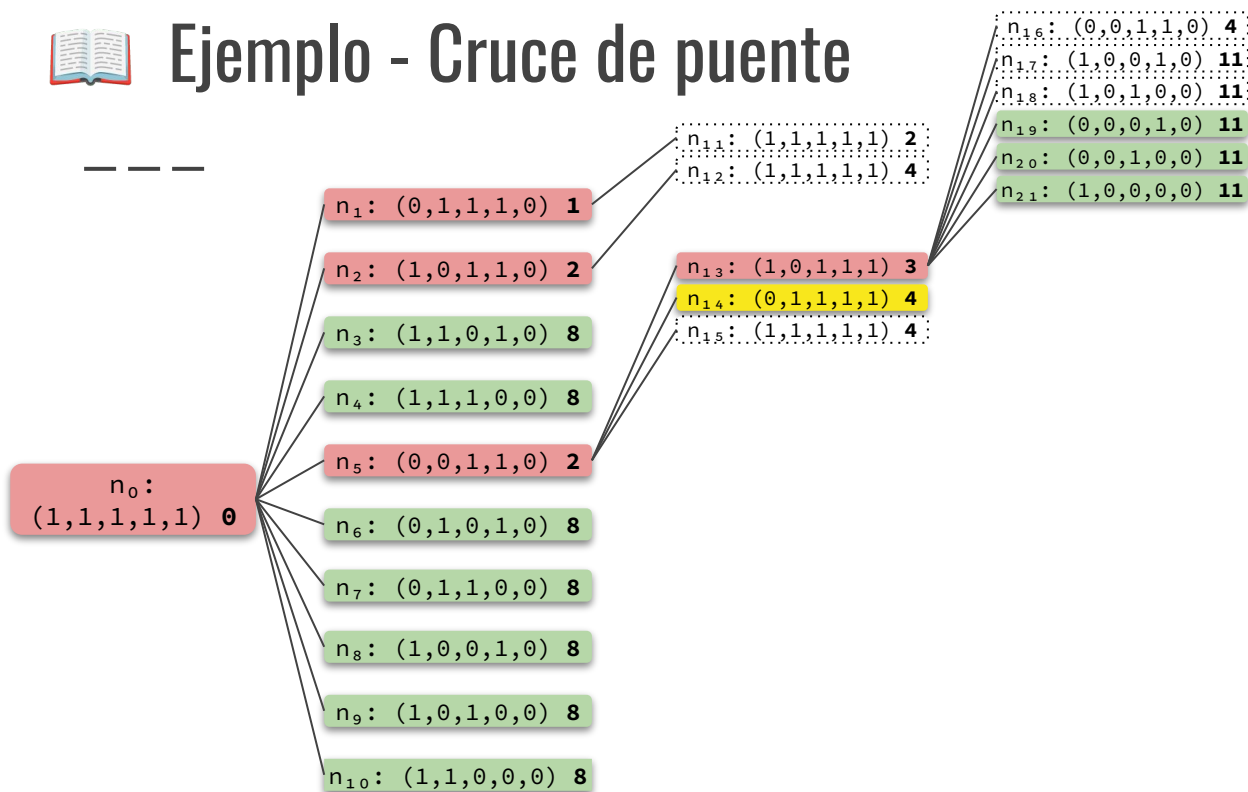
Los nodos n_{16} , n_{17} y n_{18} contienen estados ya alcanzados en el nivel 1 y con un **mayor costo de camino**, por lo tanto también los descartamos.

FRONTERA = $\{n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{14}, n_{19}, n_{20}, n_{21}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11\}$



Ejemplo - Cruce de puente

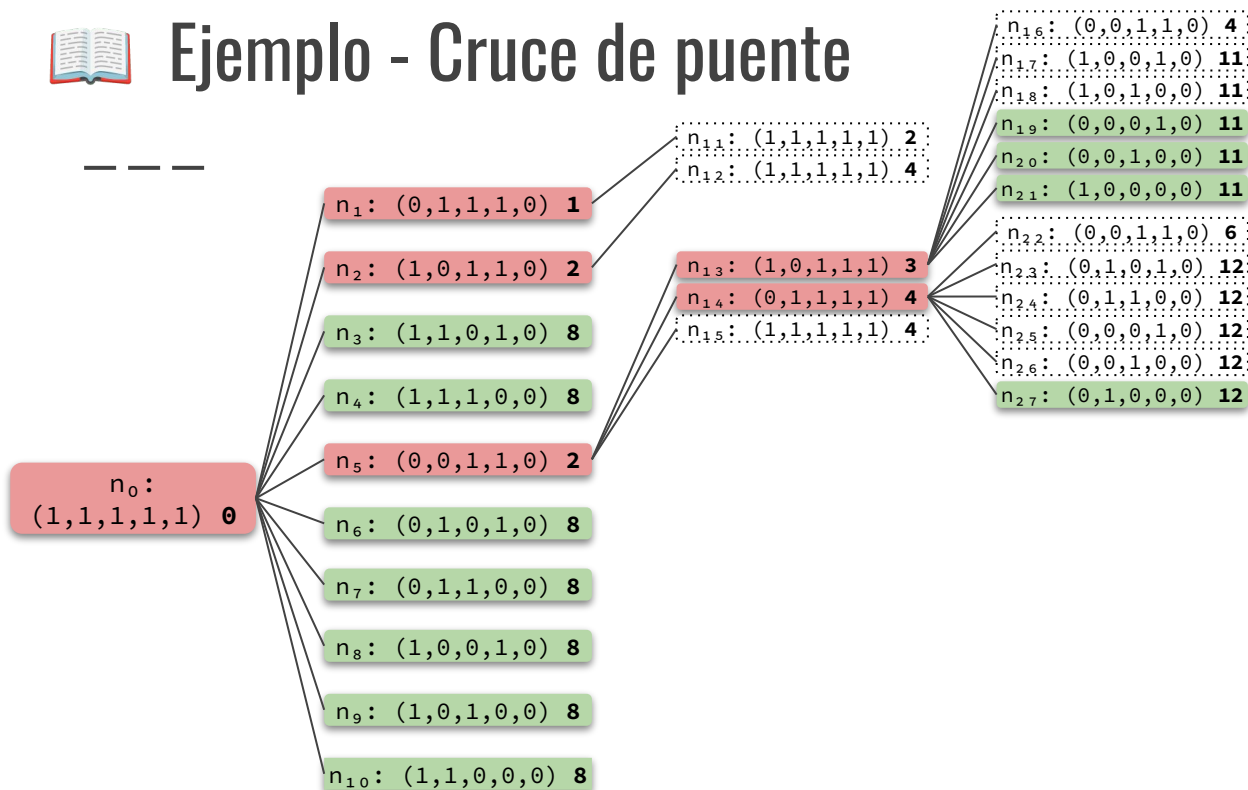


FRONTERA = $\{n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11\}$



Ejemplo - Cruce de puente

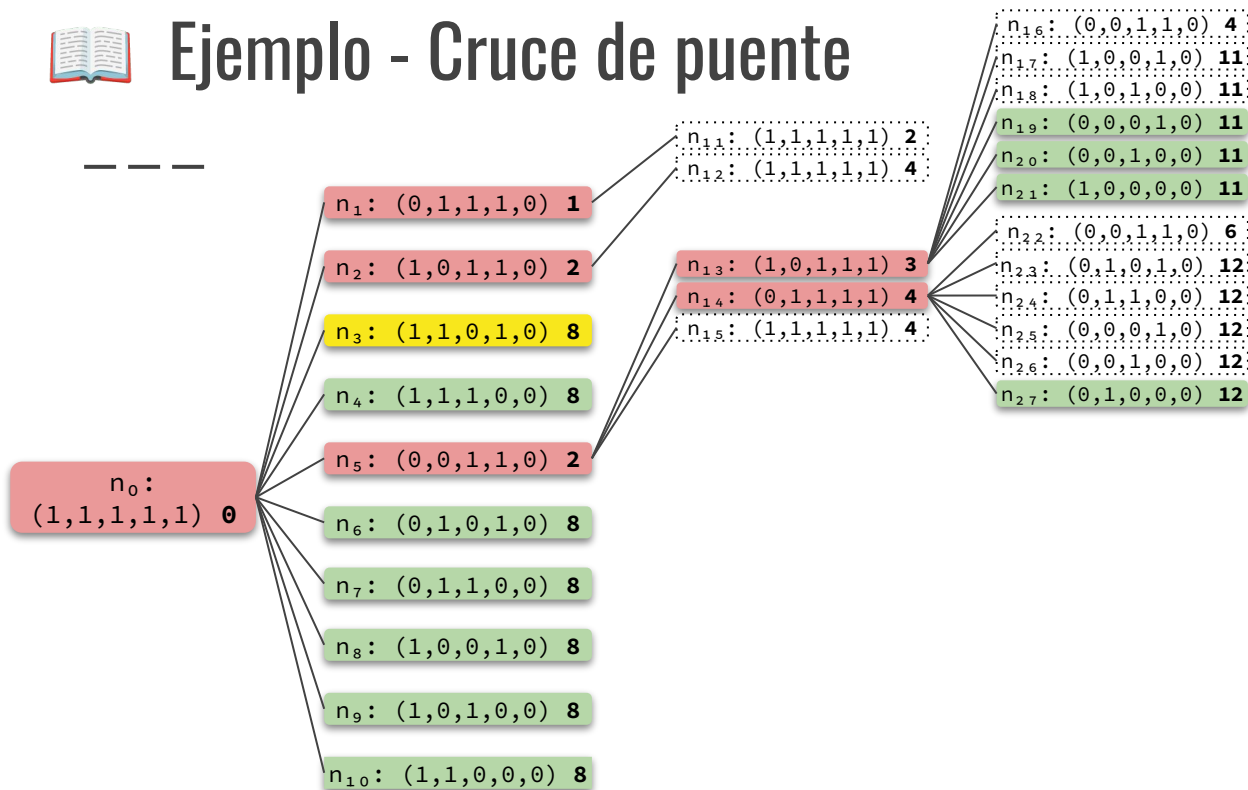


FRONTERA = $\{n_3, n_4, n_6, n_7, n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12\}$



Ejemplo - Cruce de puente

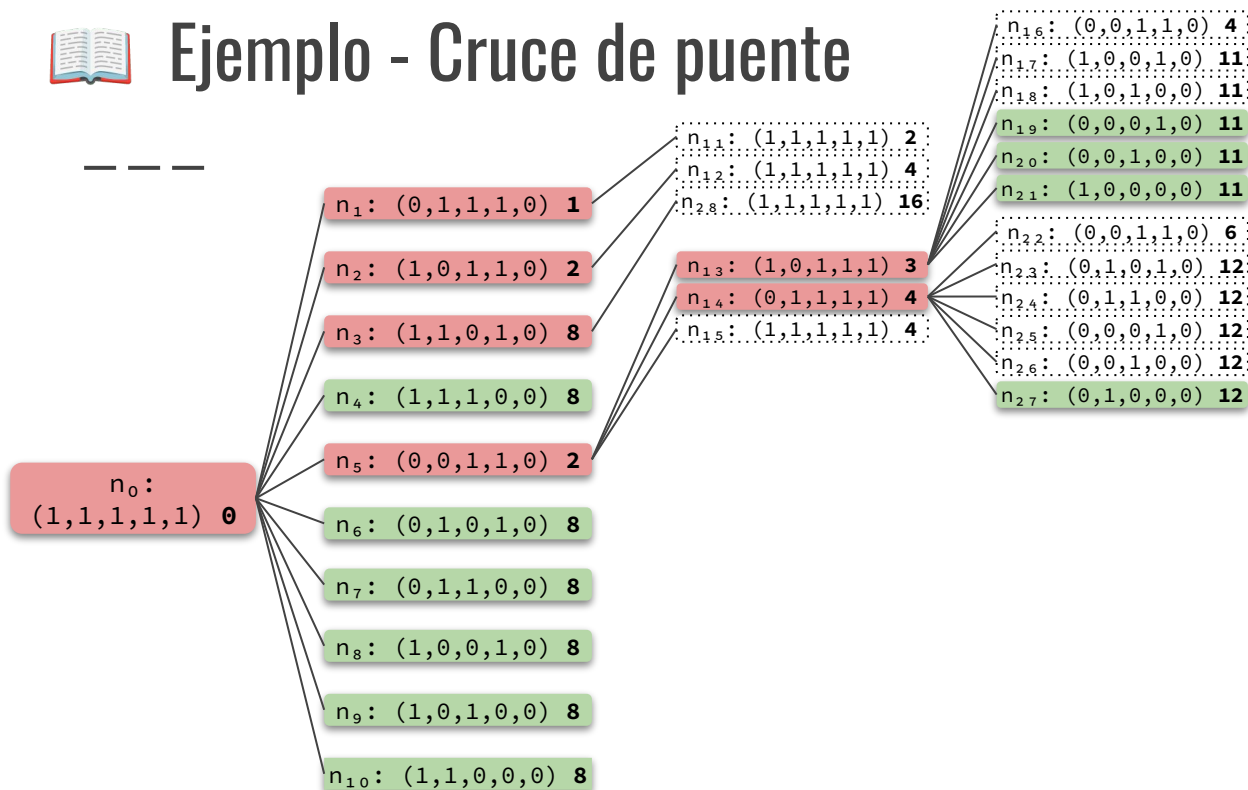


FRONTERA = $\{n_4, n_6, n_7, n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12\}$



Ejemplo - Cruce de puente

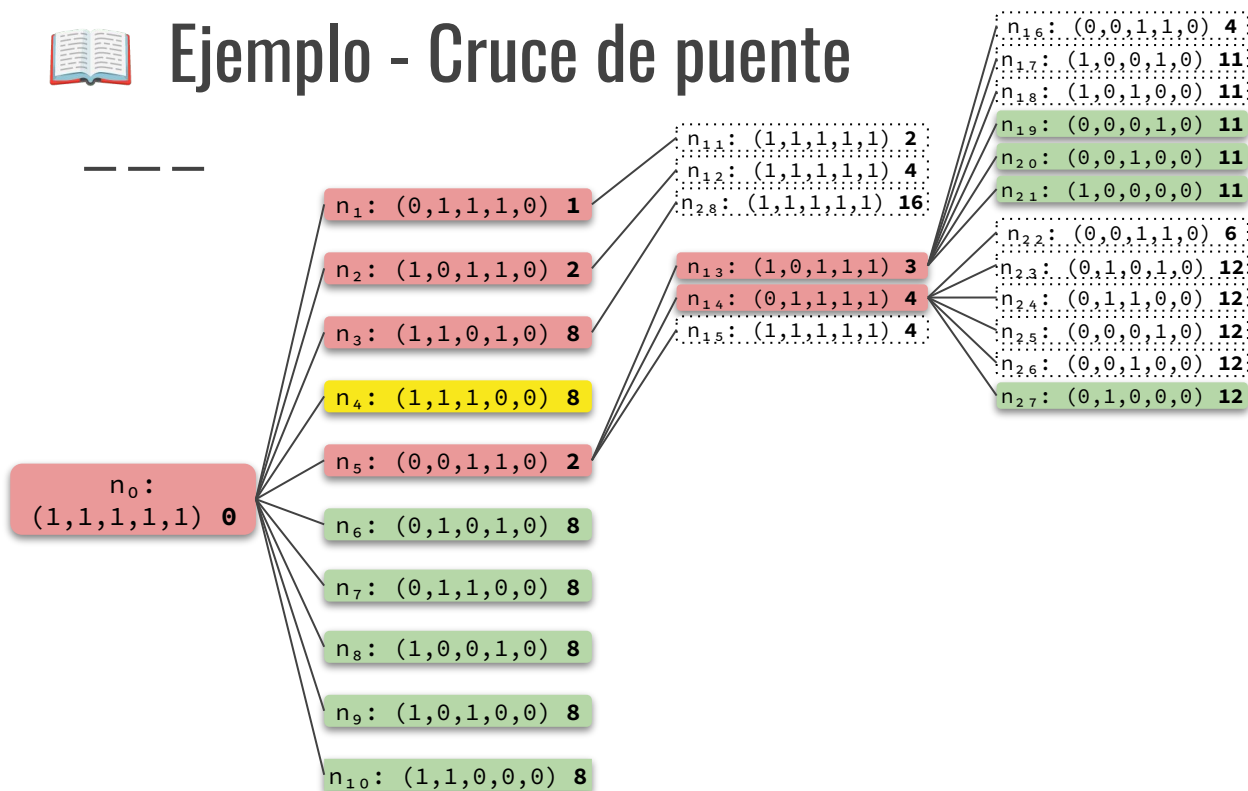


FRONTERA = $\{n_4, n_6, n_7, n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12\}$



Ejemplo - Cruce de puente

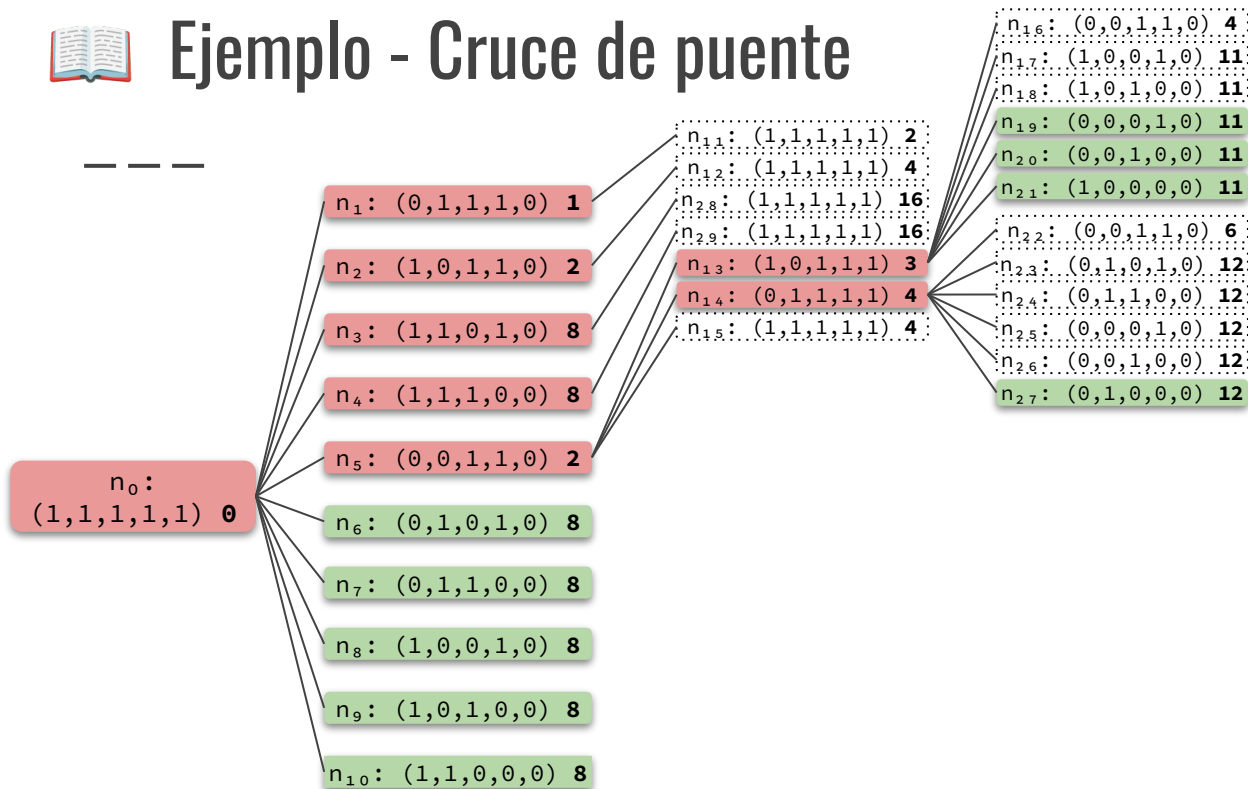


FRONTERA = $\{n_6, n_7, n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12\}$



Ejemplo - Cruce de puente

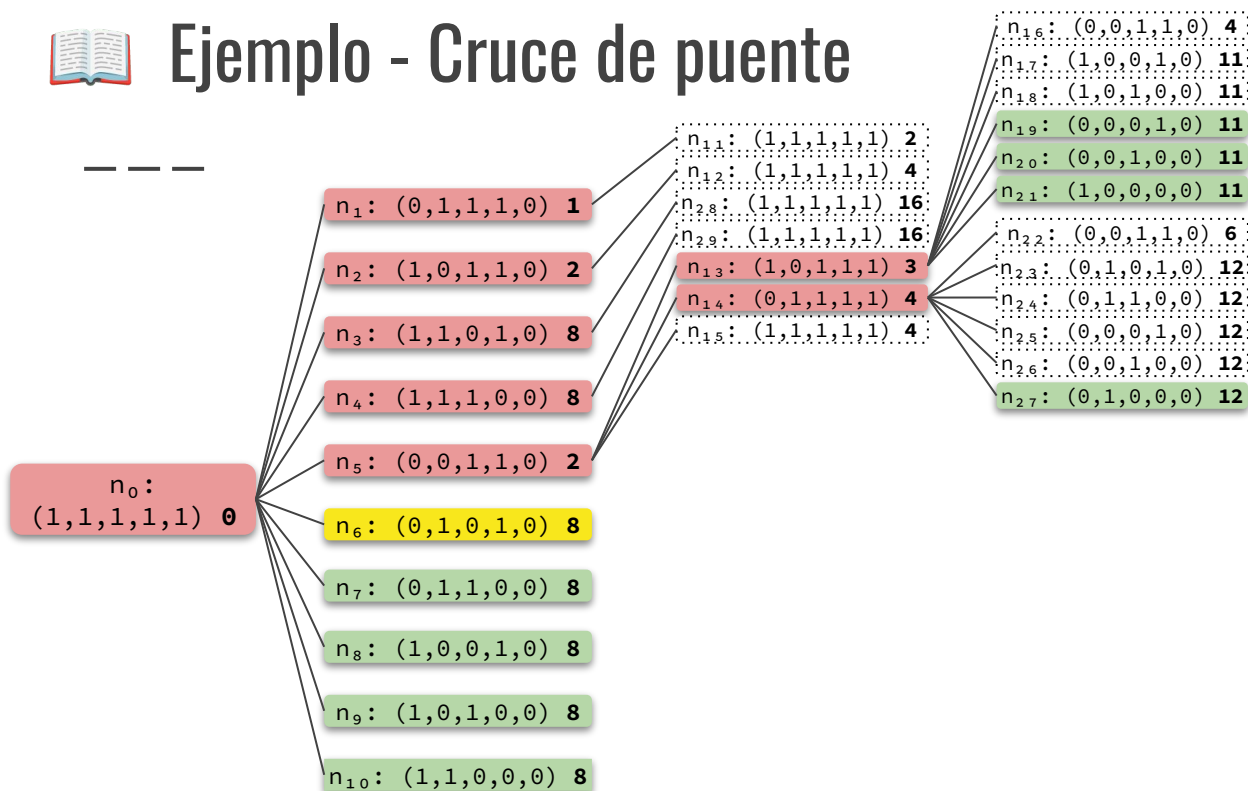


FRONTERA = $\{n_6, n_7, n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12\}$



Ejemplo - Cruce de puente

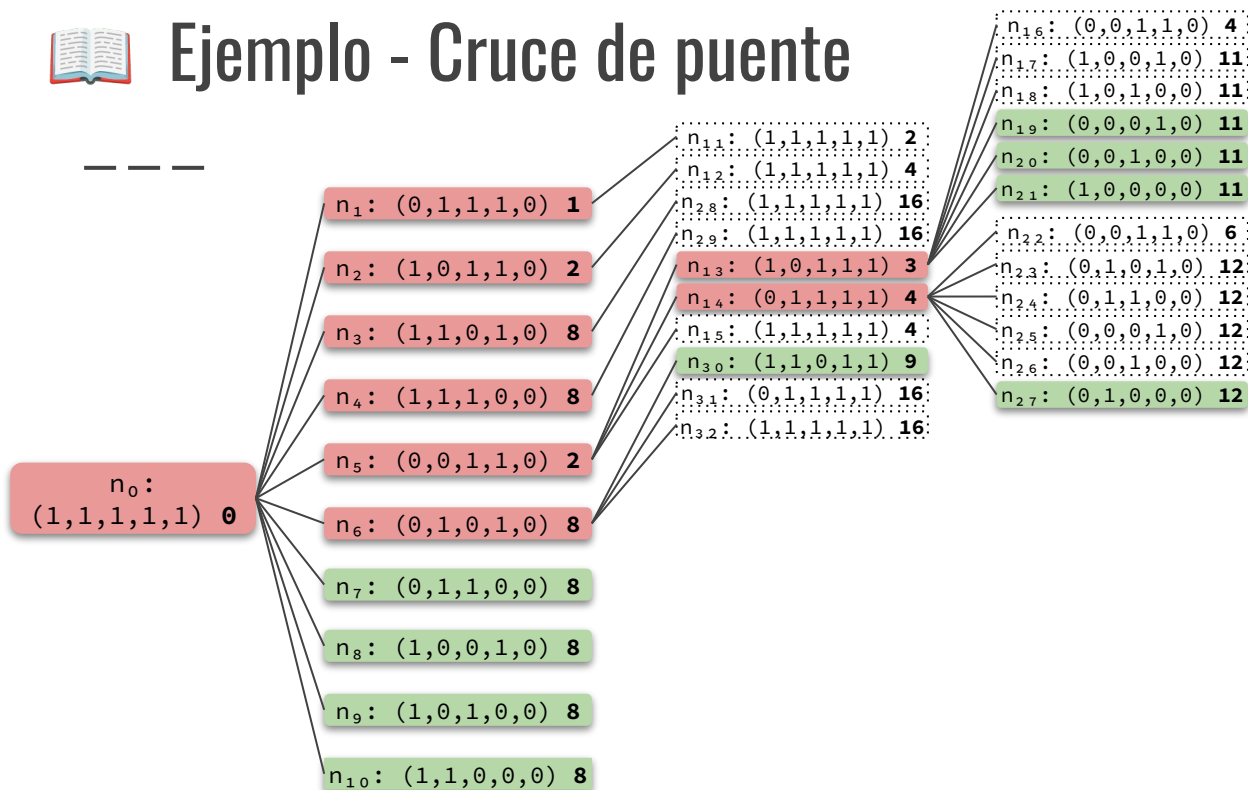


FRONTERA = $\{n_7, n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12\}$



Ejemplo - Cruce de puente

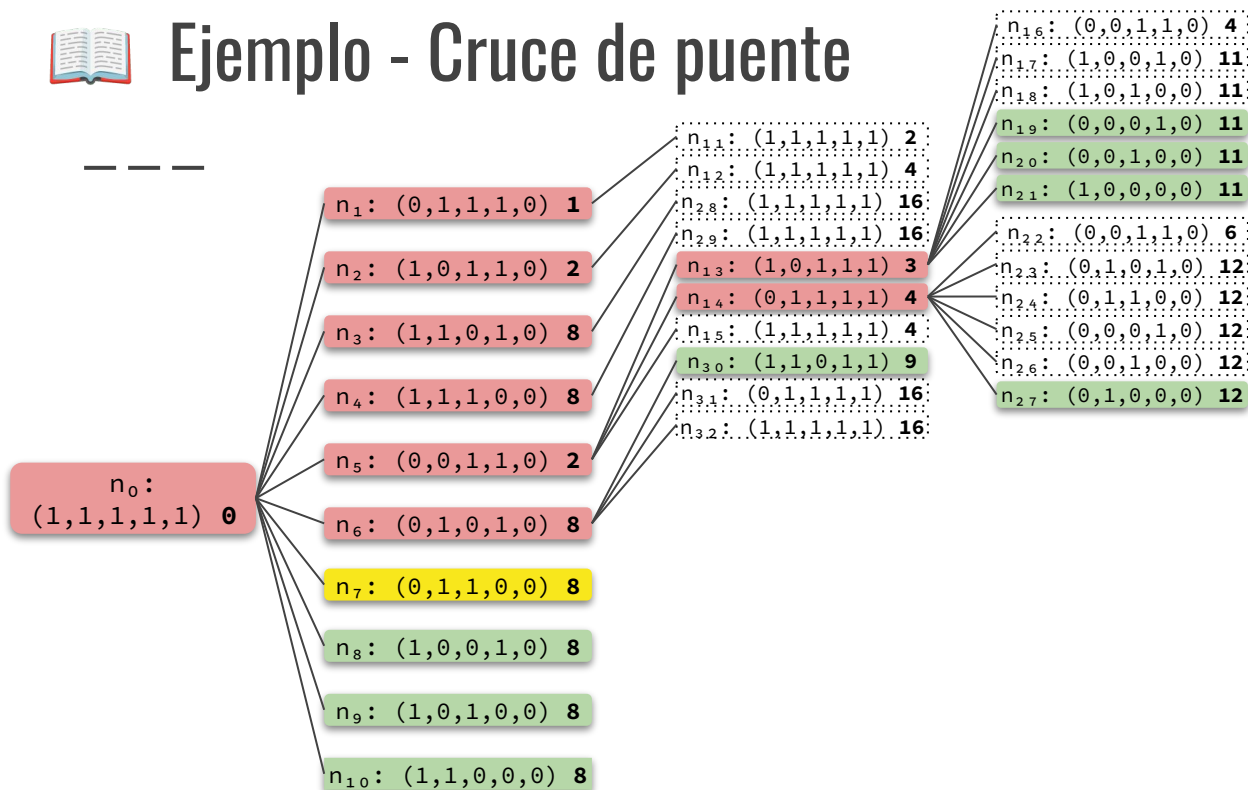


FRONTERA = $\{n_7, n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}, n_{30}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9\}$



Ejemplo - Cruce de puente

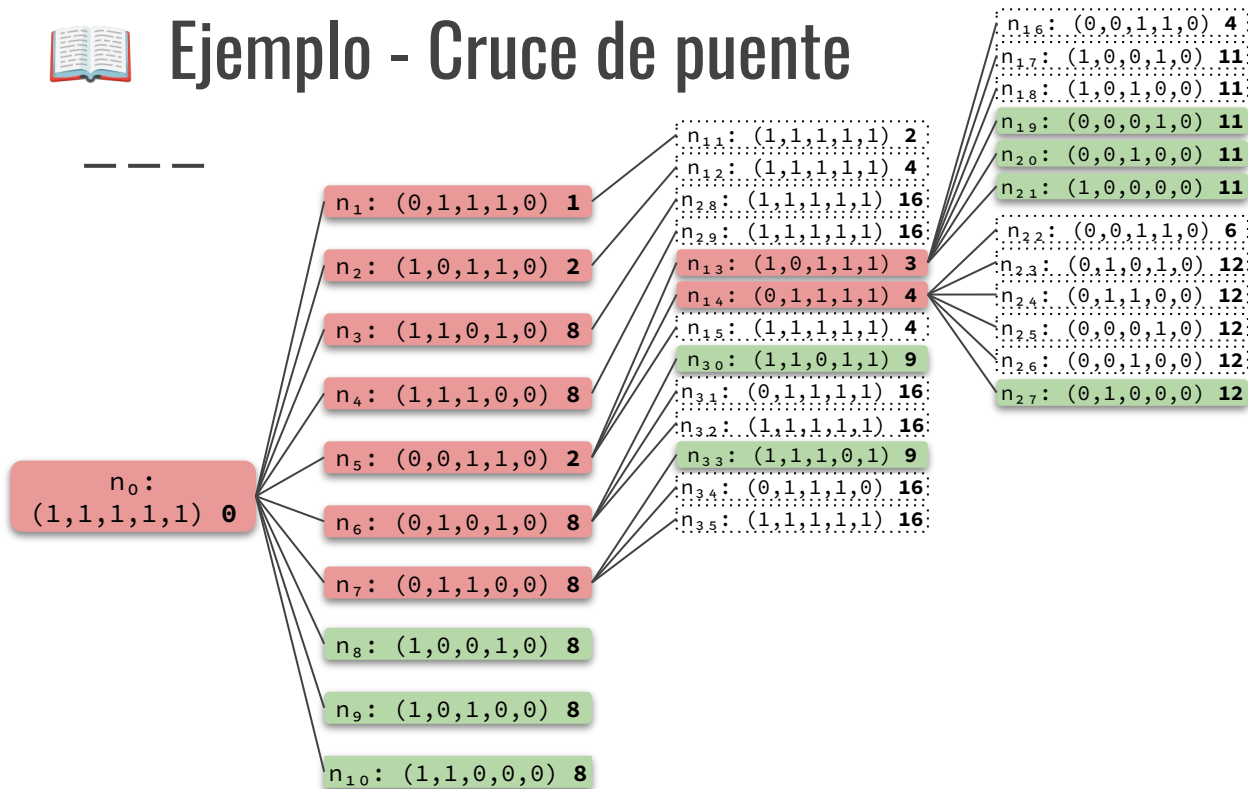


FRONTERA = $\{n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}, n_{30}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9\}$



Ejemplo - Cruce de puente

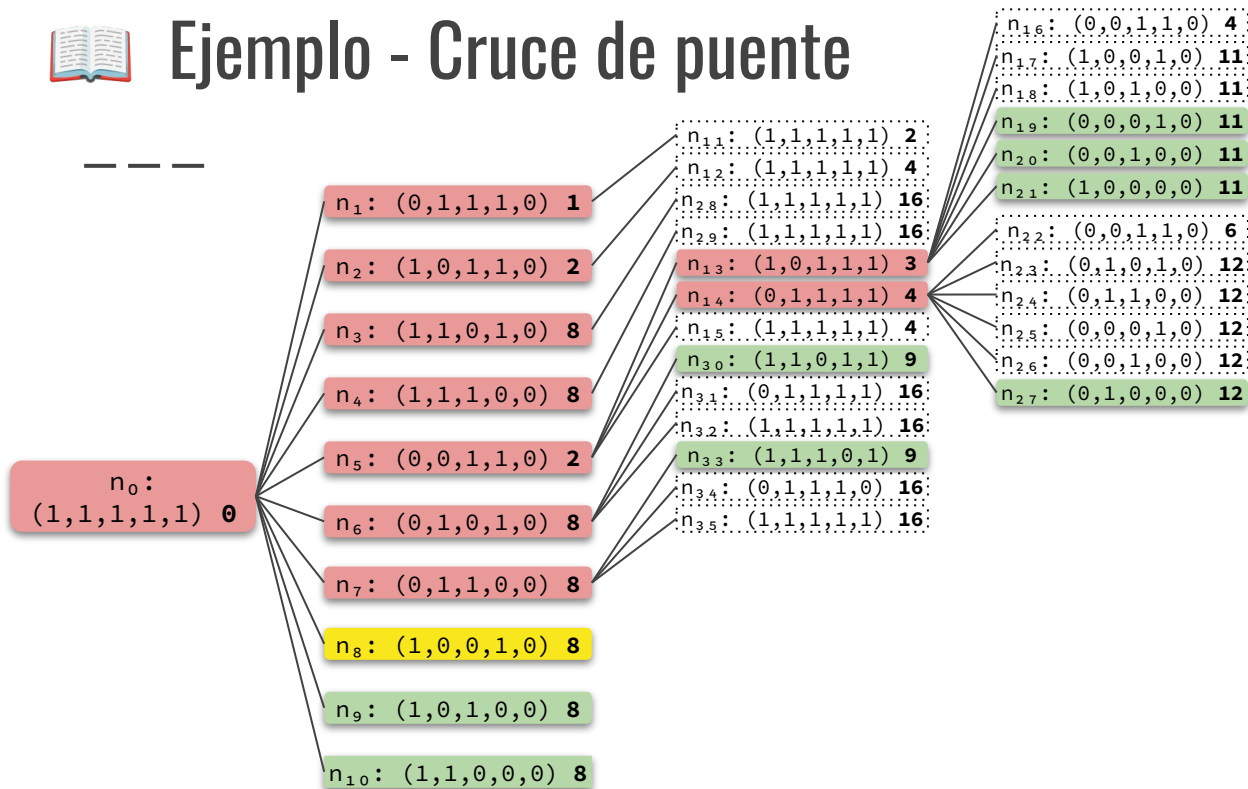


FRONTERA = $\{n_8, n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}, n_{30}, n_{33}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente



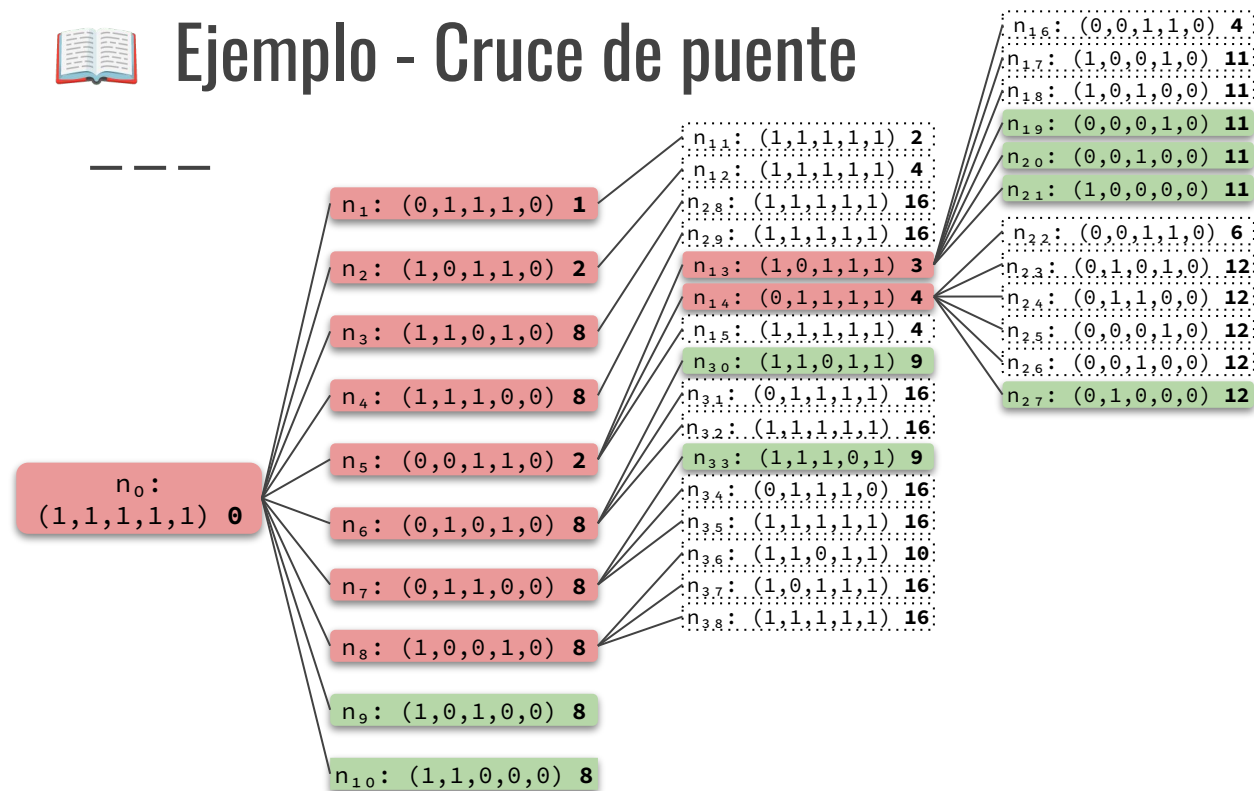
FRONTERA = $\{n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}, n_{30}, n_{33}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente

— — —



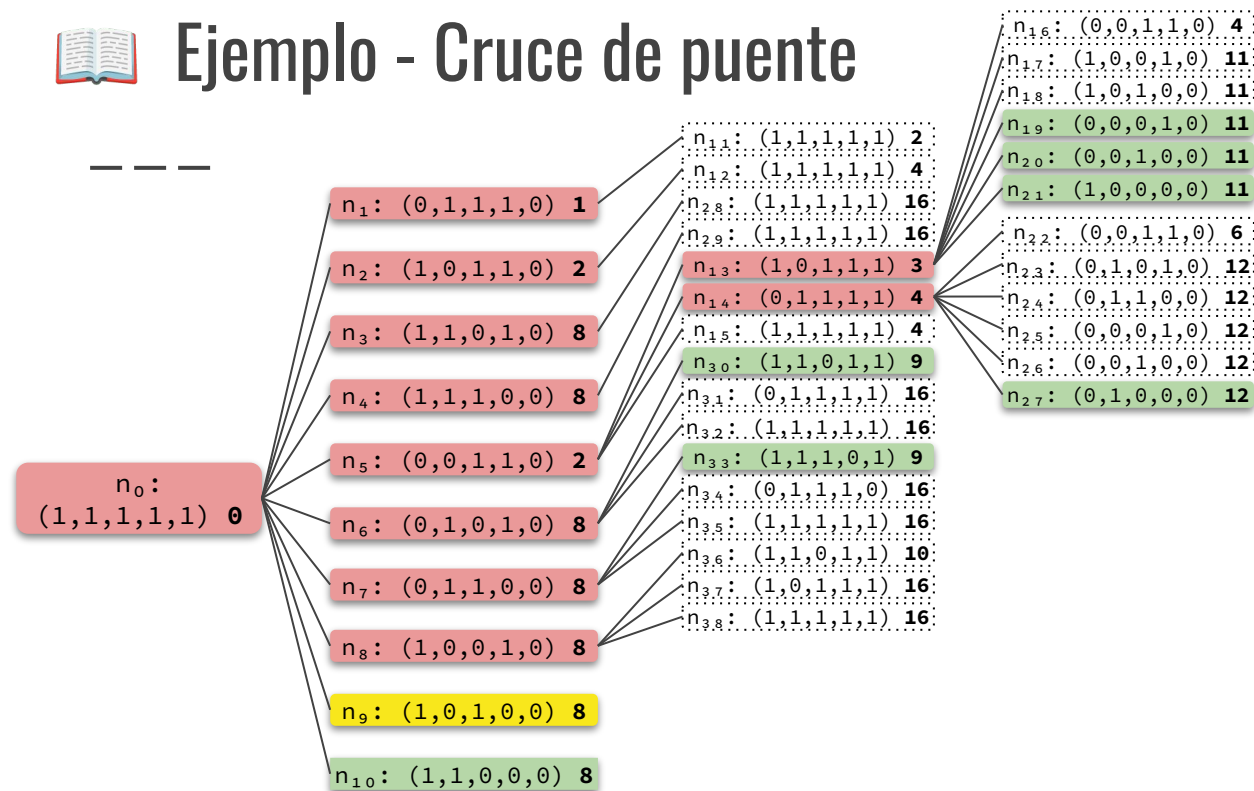
FRONTERA = $\{n_9, n_{10}, n_{19}, n_{20}, n_{21}, n_{27}, n_{30}, n_{33}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente

— — —



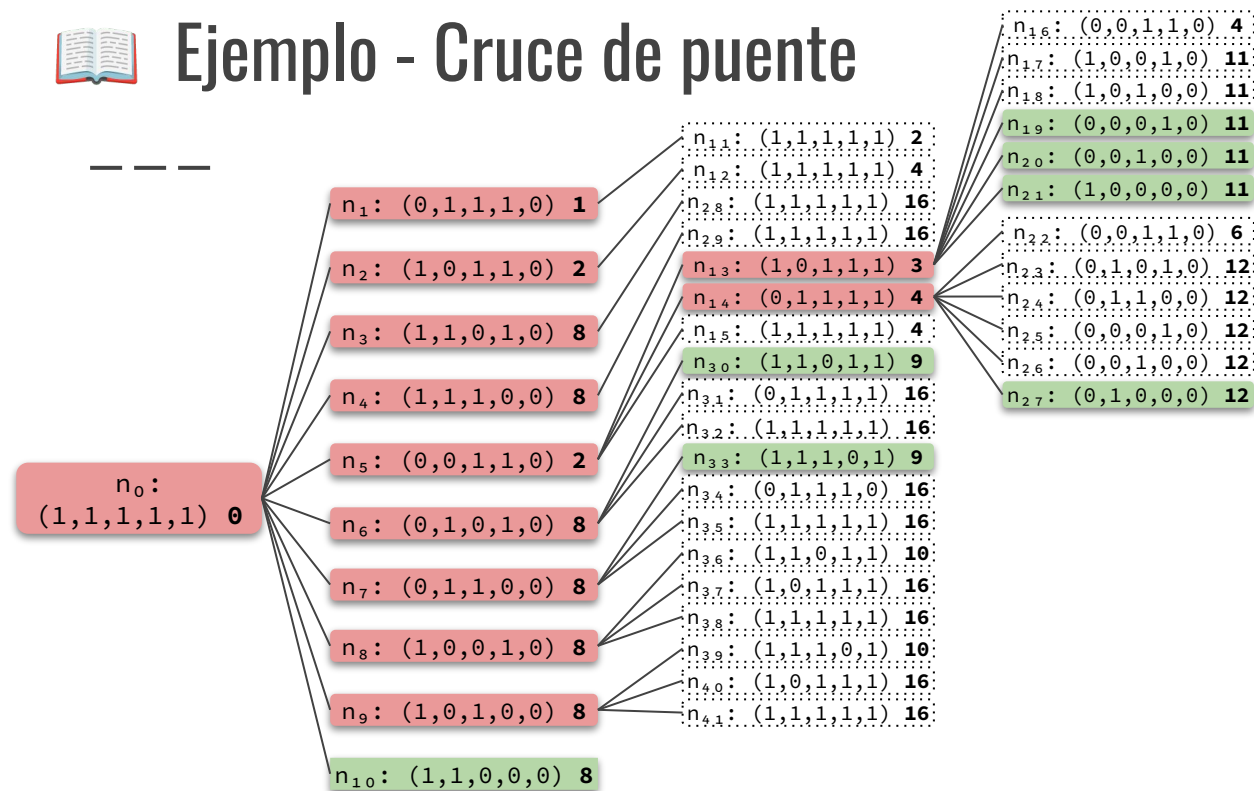
FRONTERA = $\{n_{10}, n_{19}, n_{20}, n_{21}, n_{27}, n_{30}, n_{33}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente

— — —



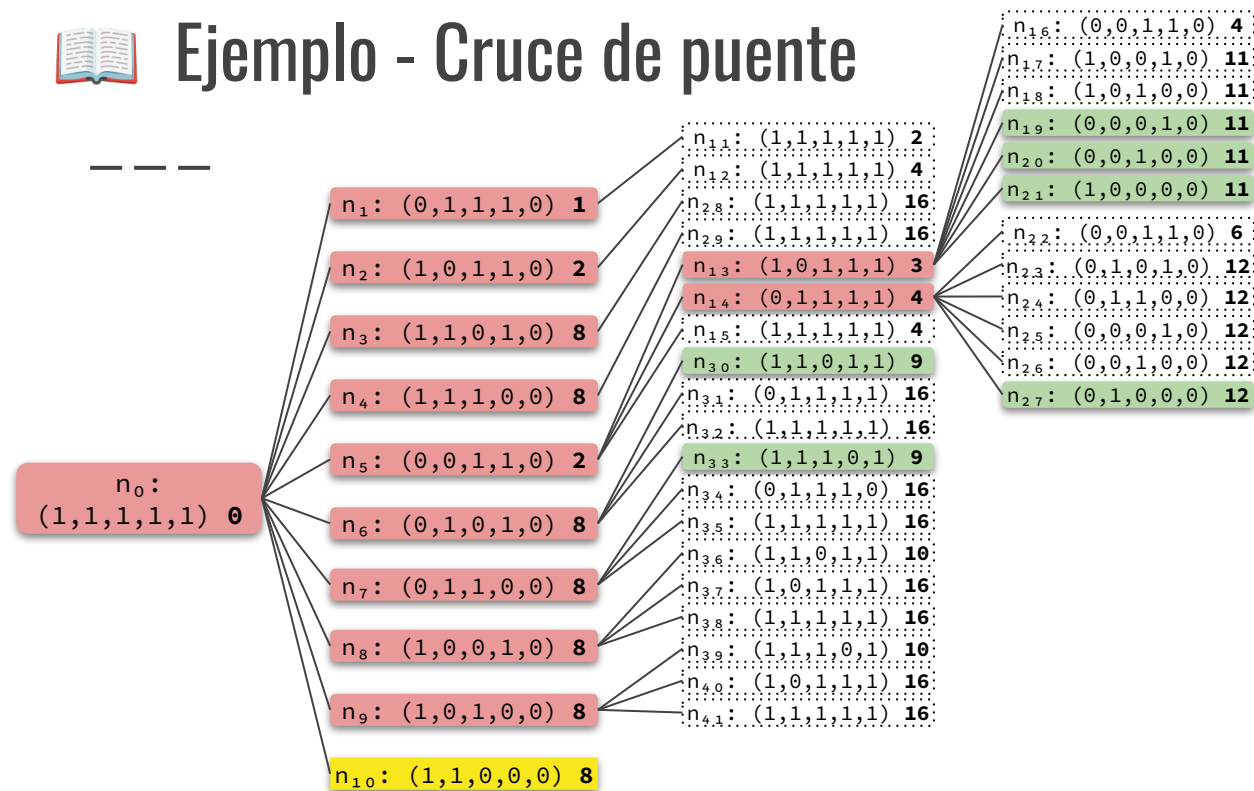
FRONTERA = $\{n_{10}, n_{19}, n_{20}, n_{21}, n_{27}, n_{30}, n_{33}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente

— — —

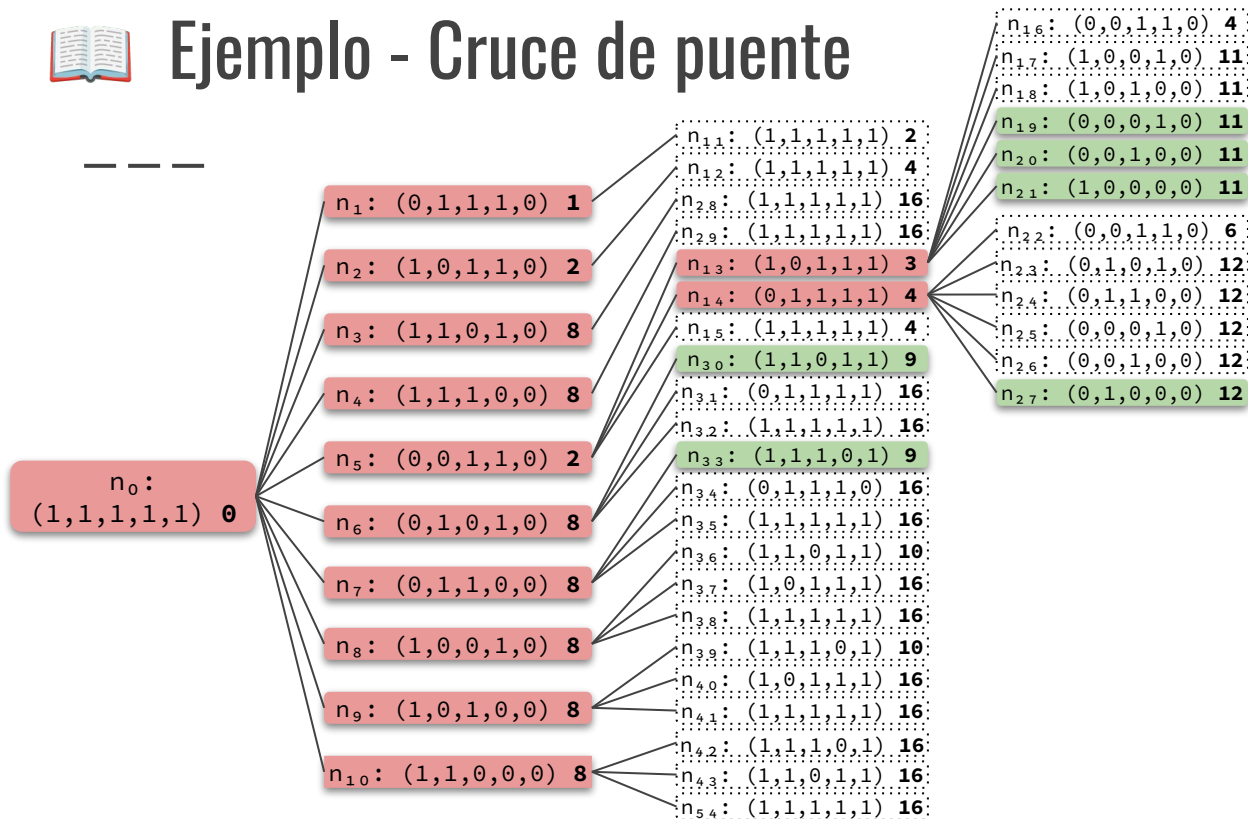


FRONTERA = $\{n_{19}, n_{20}, n_{21}, n_{27}, n_{30}, n_{33}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente

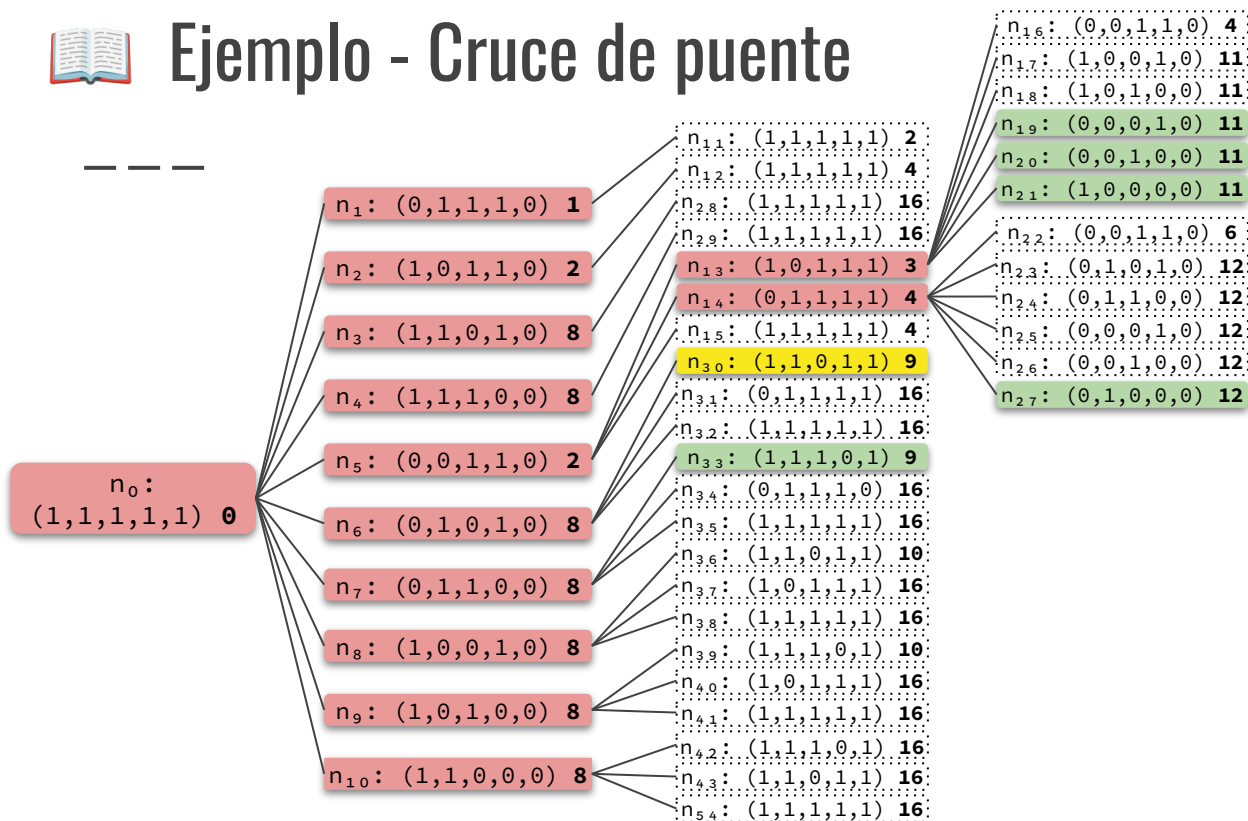


FRONTERA = $\{n_{19}, n_{20}, n_{21}, n_{27}, n_{30}, n_{33}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente

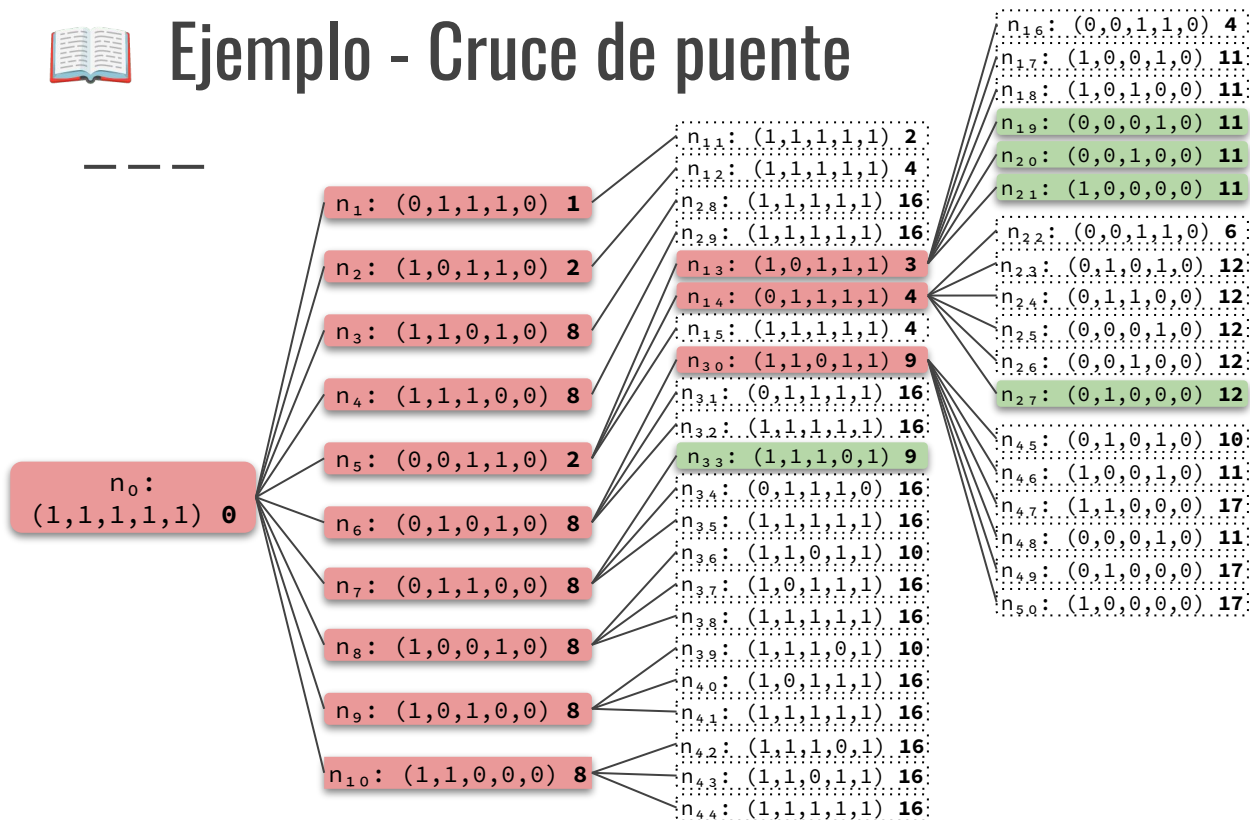


FRONTERA = $\{n_{19}, n_{20}, n_{21}, n_{27}, n_{33}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente

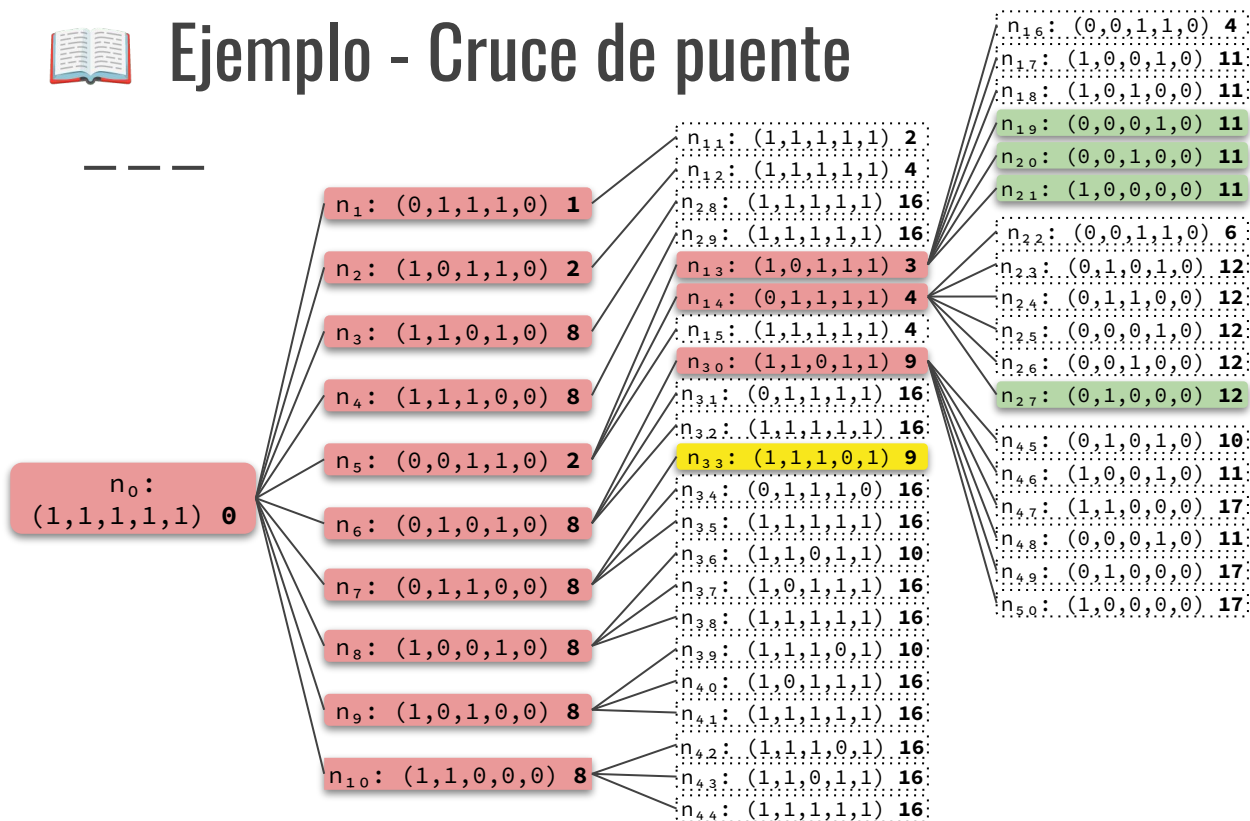


FRONTERA = $\{n_{19}, n_{20}, n_{21}, n_{27}, n_{33}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente



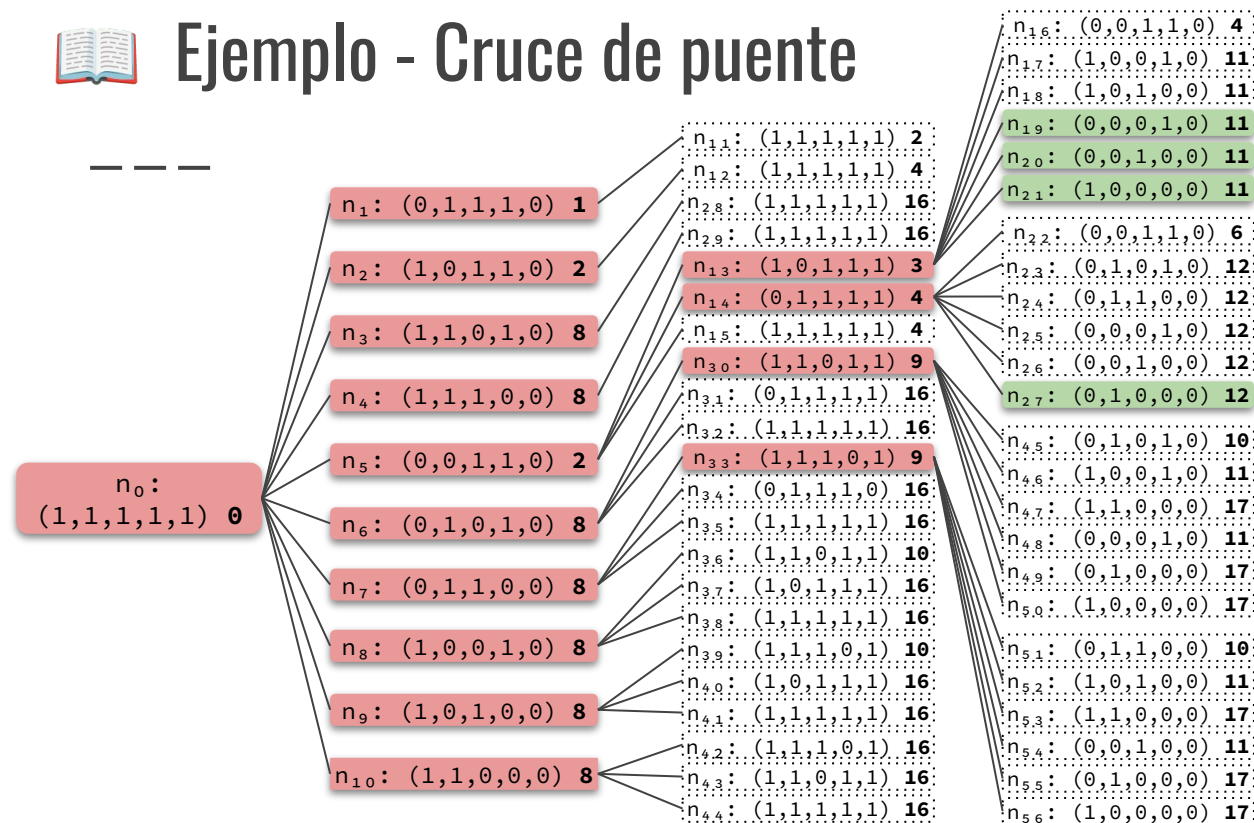
FRONTERA = $\{n_{19}, n_{20}, n_{21}, n_{27}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente

— — —

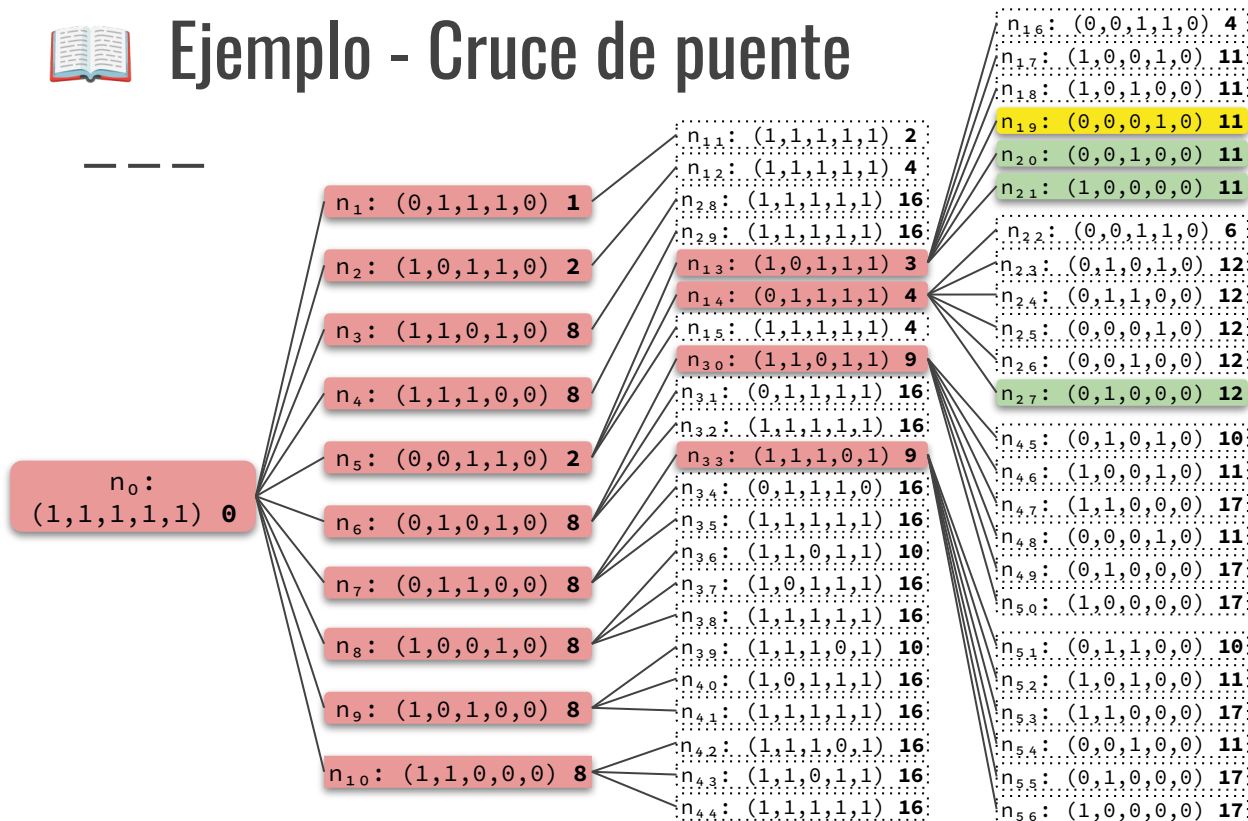


FRONTERA = $\{n_{19}, n_{20}, n_{21}, n_{27}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente



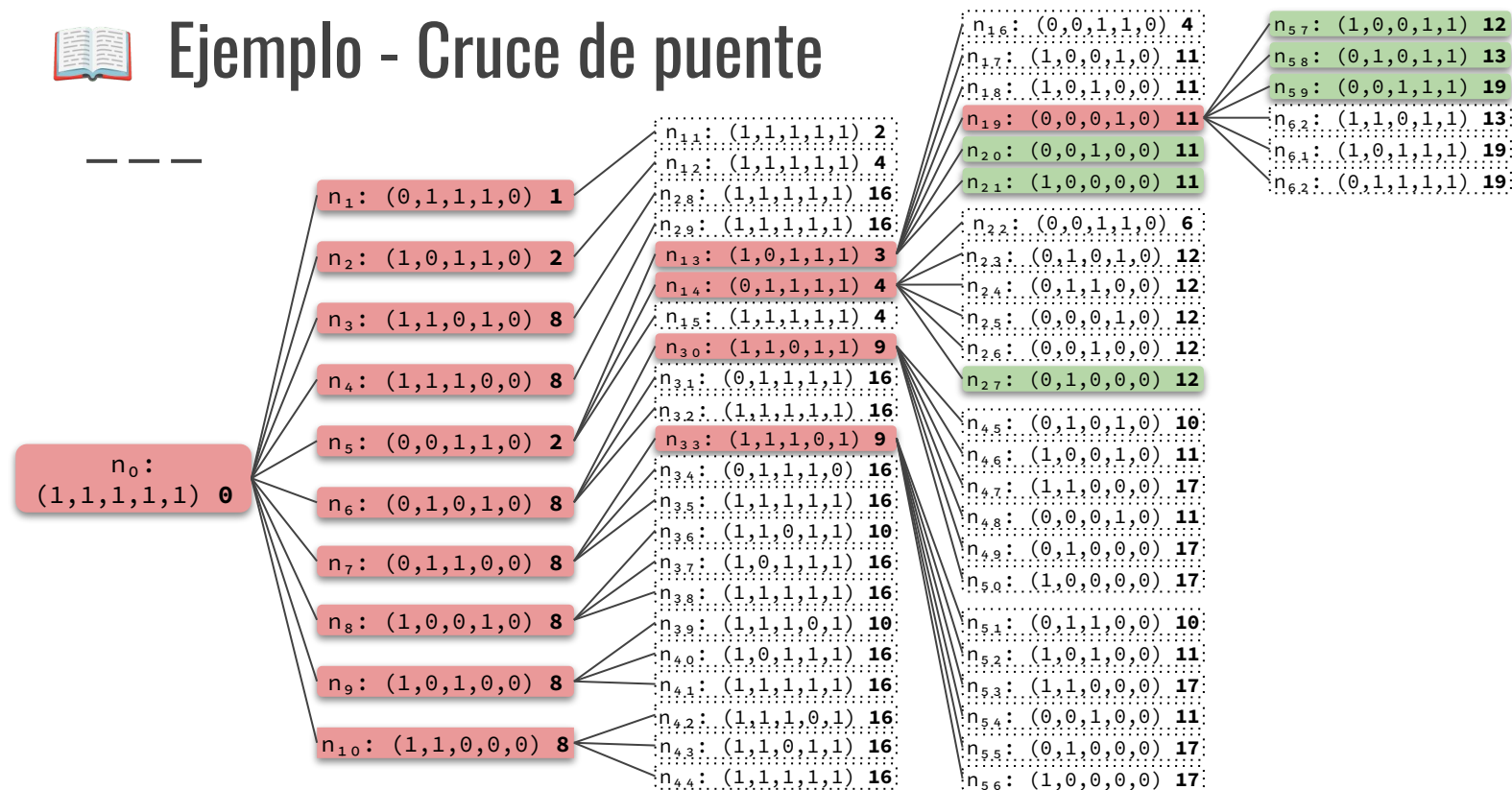
FRONTERA = $\{n_{20}, n_{21}, n_{27}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9\}$



Ejemplo - Cruce de puente

— — —

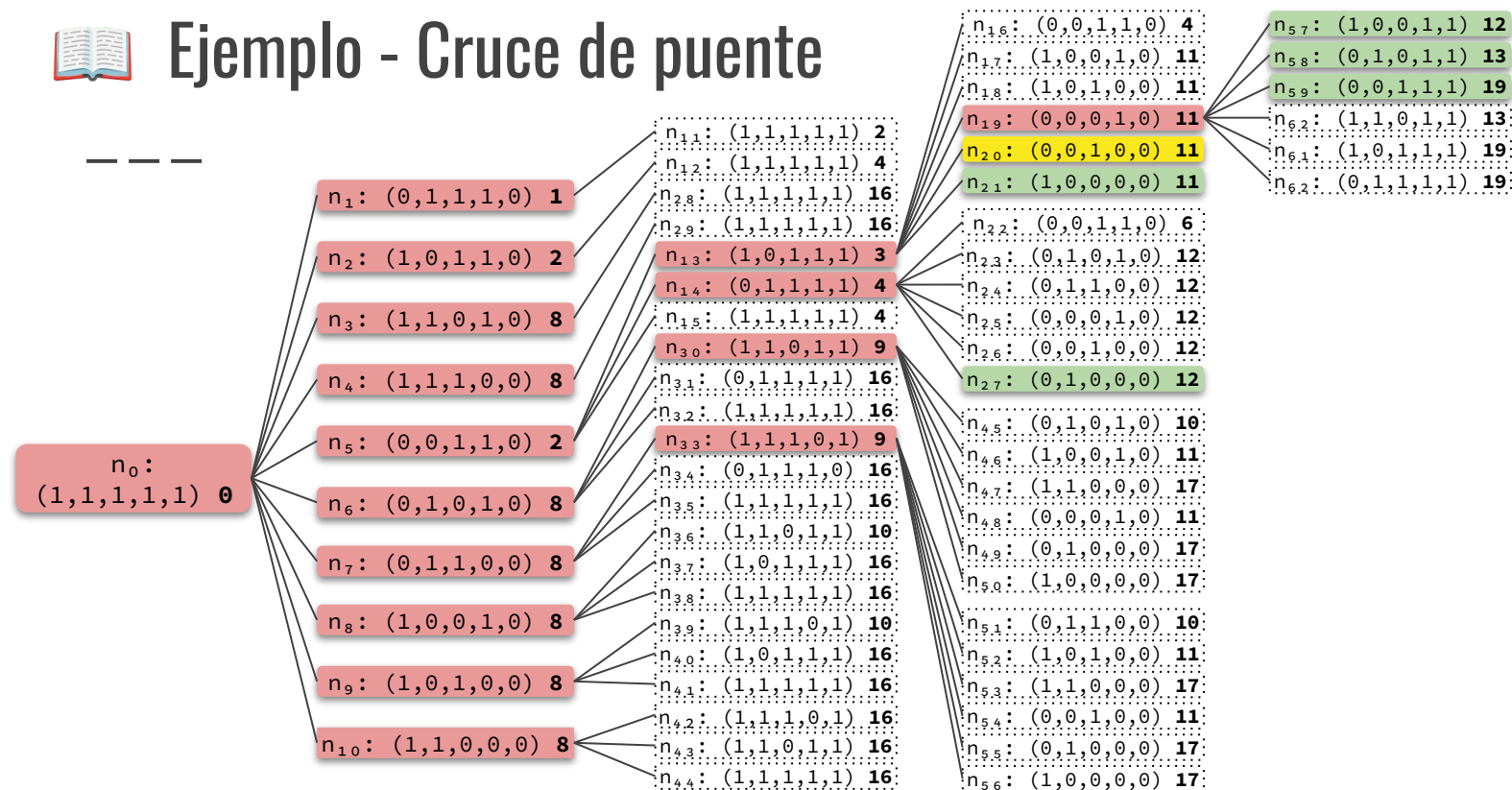


FRONTERA = $\{n_{20}, n_{21}, n_{27}, n_{57}, n_{58}, n_{59}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,0,0): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19\}$



Ejemplo - Cruce de puente

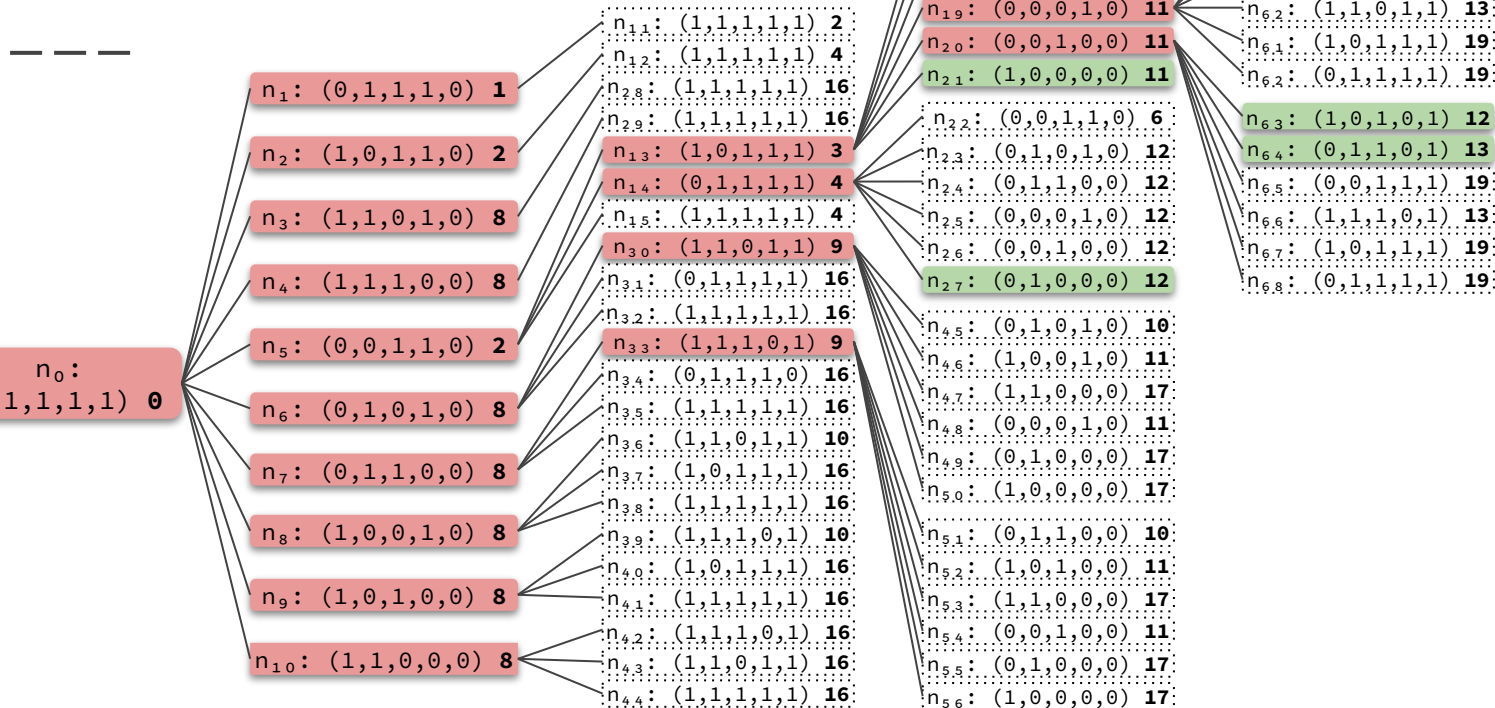


FRONTERA = $\{n_{21}, n_{27}, n_{57}, n_{58}, n_{59}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19\}$



Ejemplo - Cruce de puente

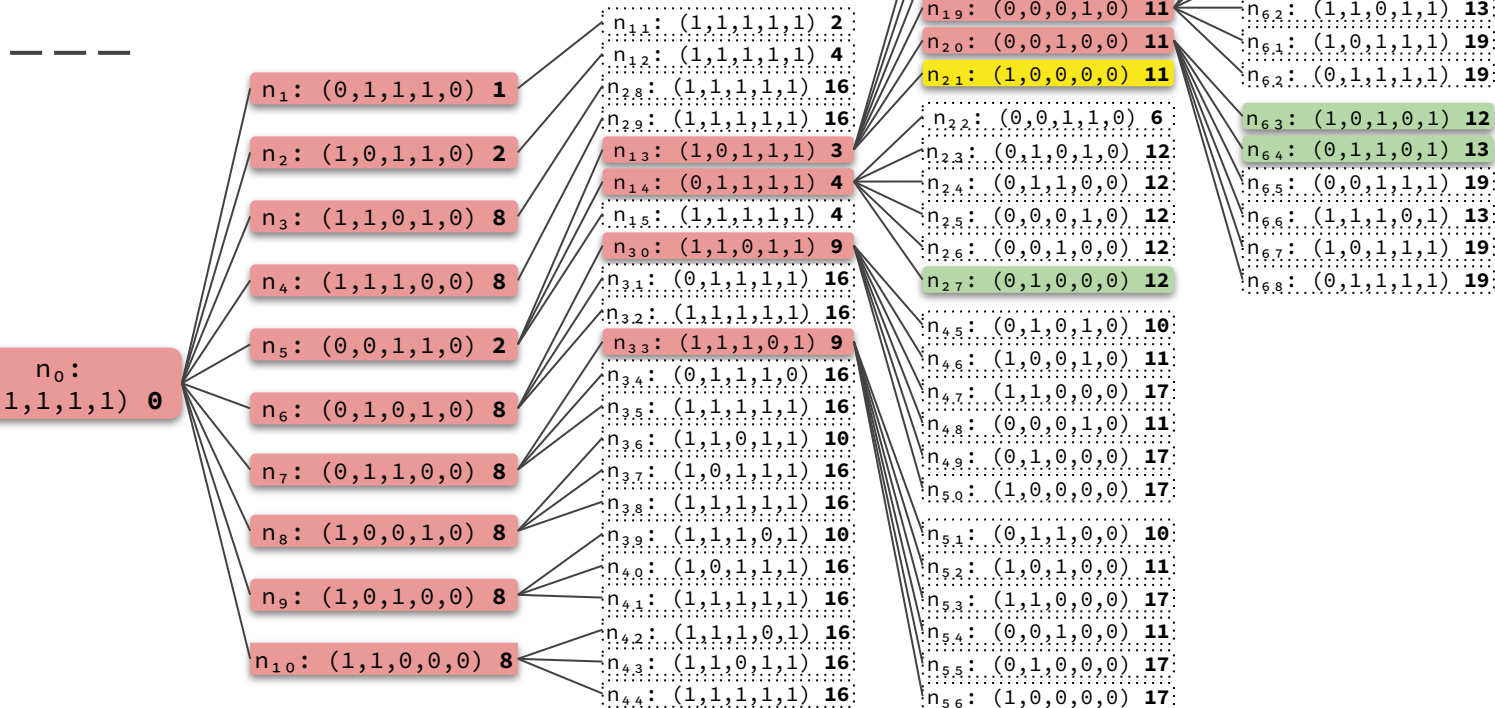


FRONTERA = $\{n_{21}, n_{27}, n_{57}, n_{58}, n_{59}, n_{63}, n_{64}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,0,0): 12, (1,0,0,0,0): 12, (0,1,0,1,1): 9, (1,1,0,1,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13\}$



Ejemplo - Cruce de puente

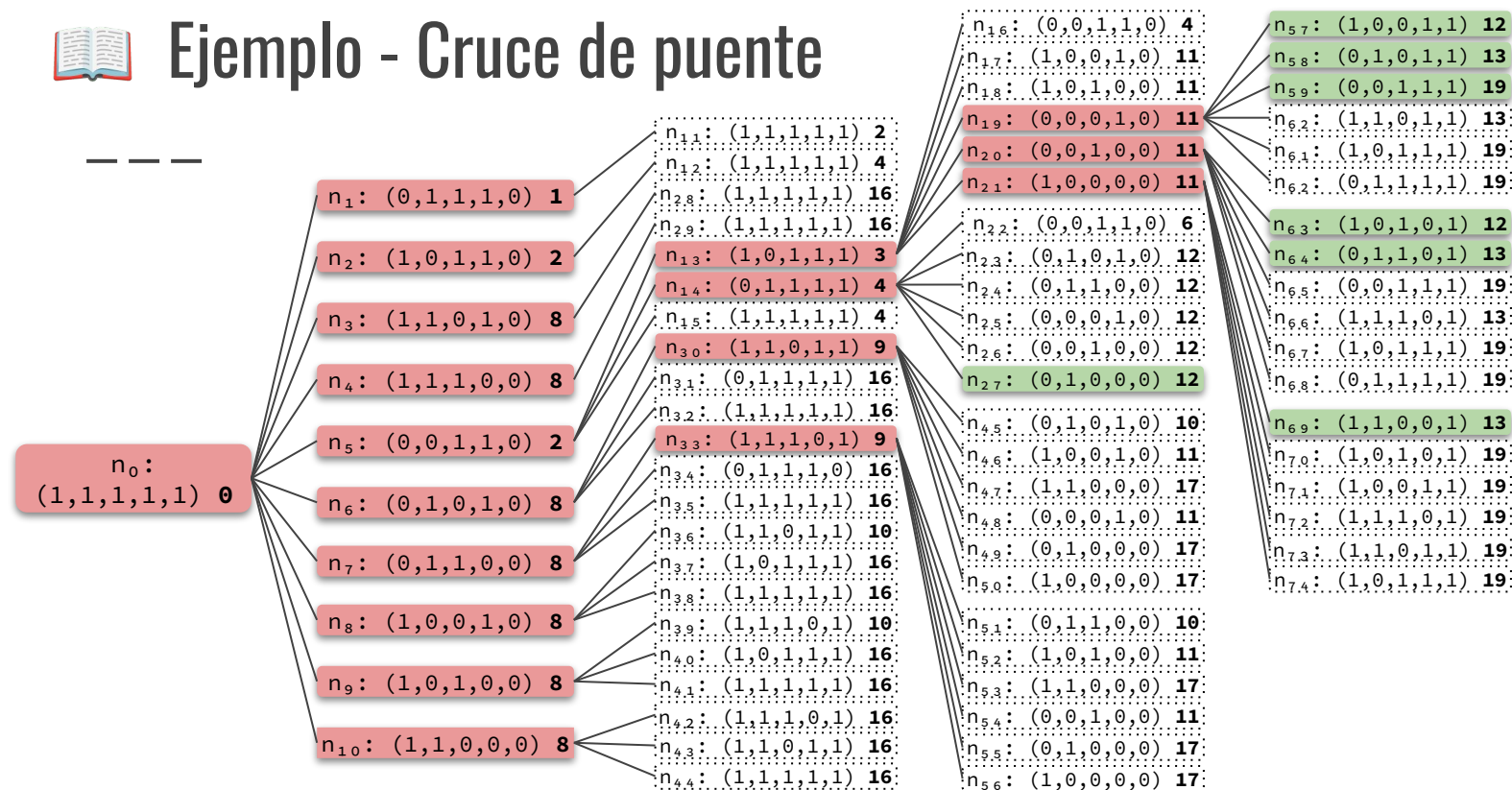


FRONTERA = $\{n_{27}, n_{57}, n_{58}, n_{59}, n_{63}, n_{64}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13\}$



Ejemplo - Cruce de puente

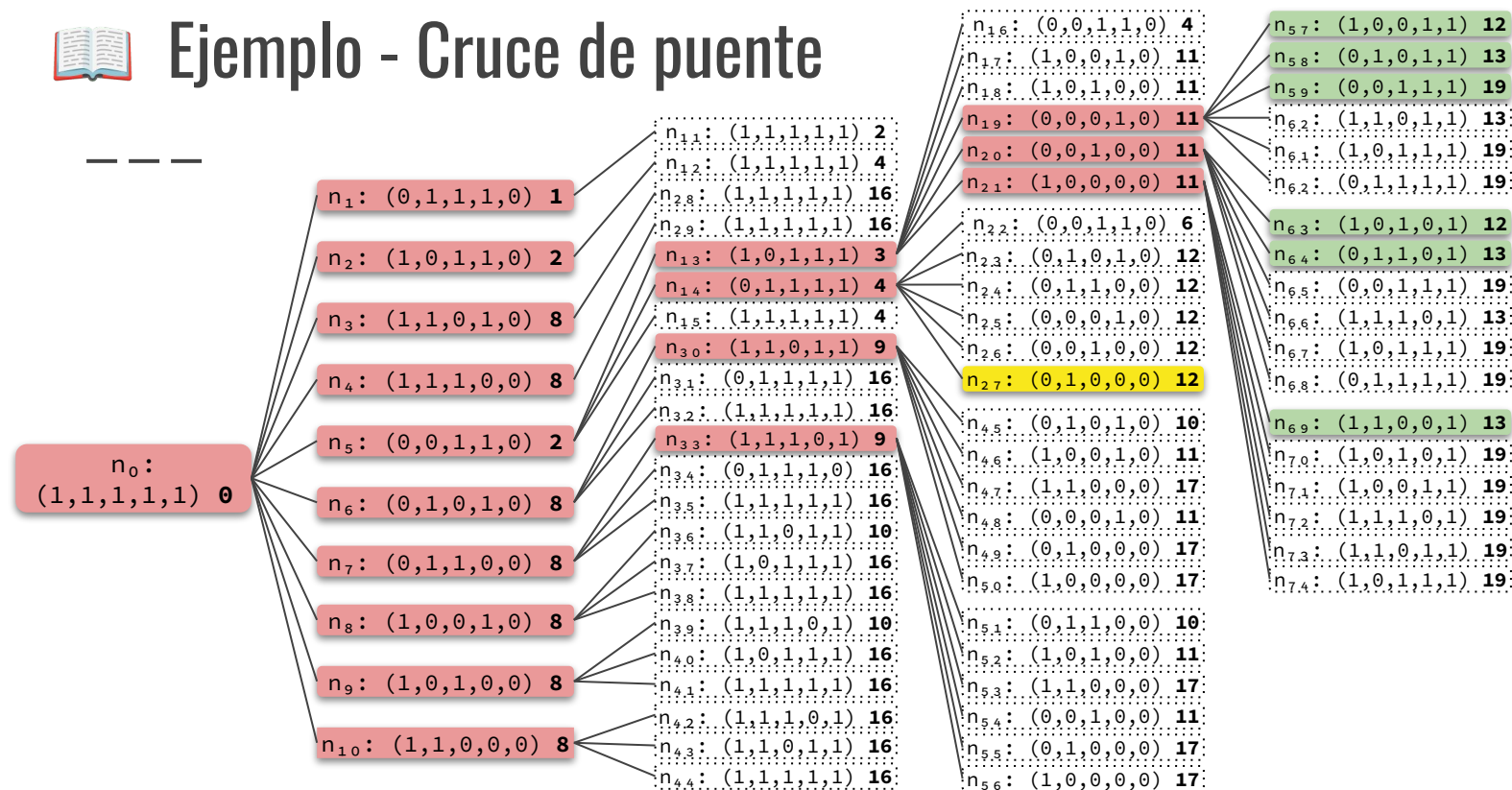


FRONTERA = $\{n_{27}, n_{57}, n_{58}, n_{59}, n_{63}, n_{64}, n_{69}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13\}$



Ejemplo - Cruce de puente

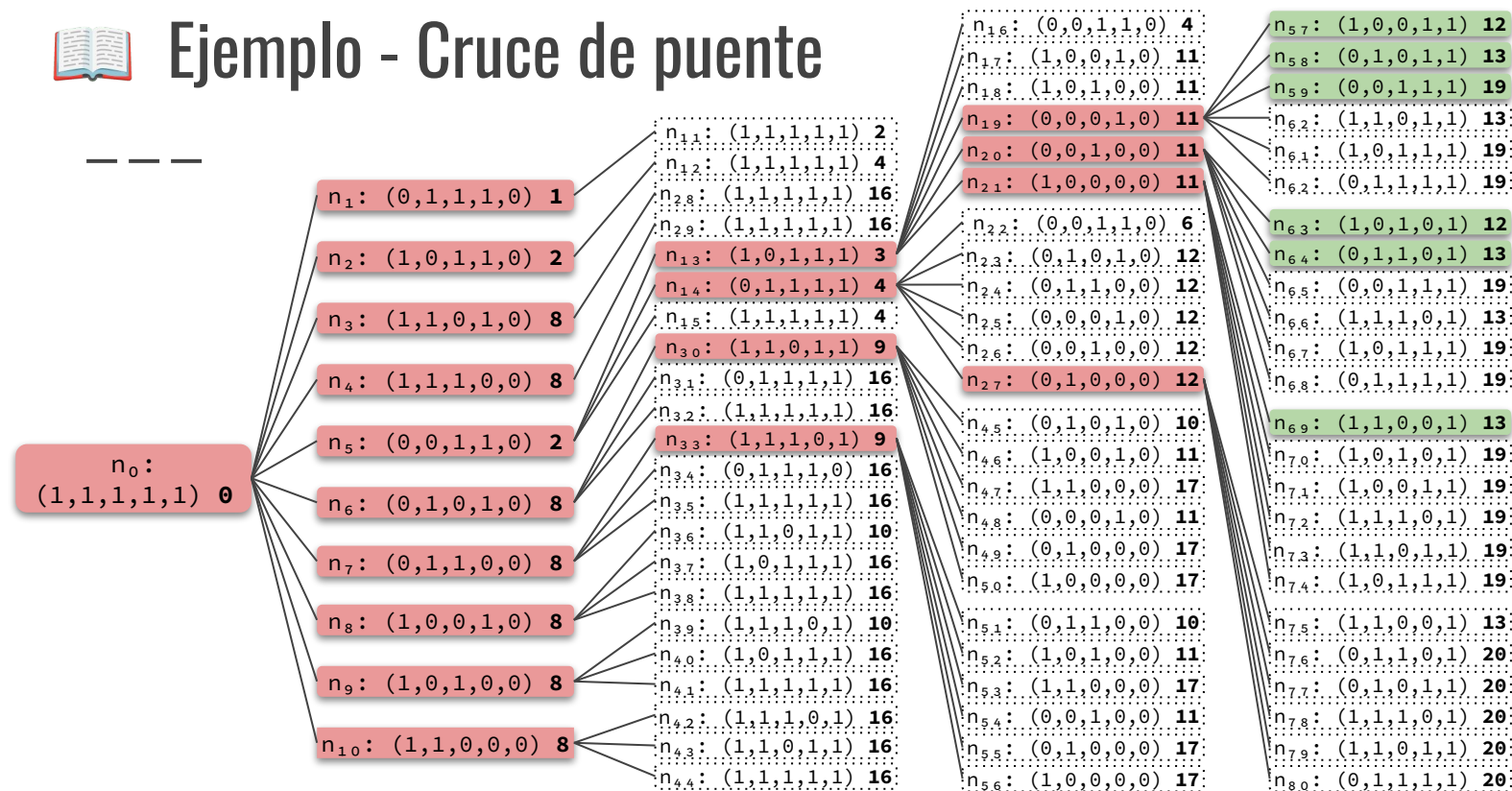


FRONTERA = $\{n_{57}, n_{58}, n_{59}, n_{63}, n_{64}, n_{69}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13\}$



Ejemplo - Cruce de puente

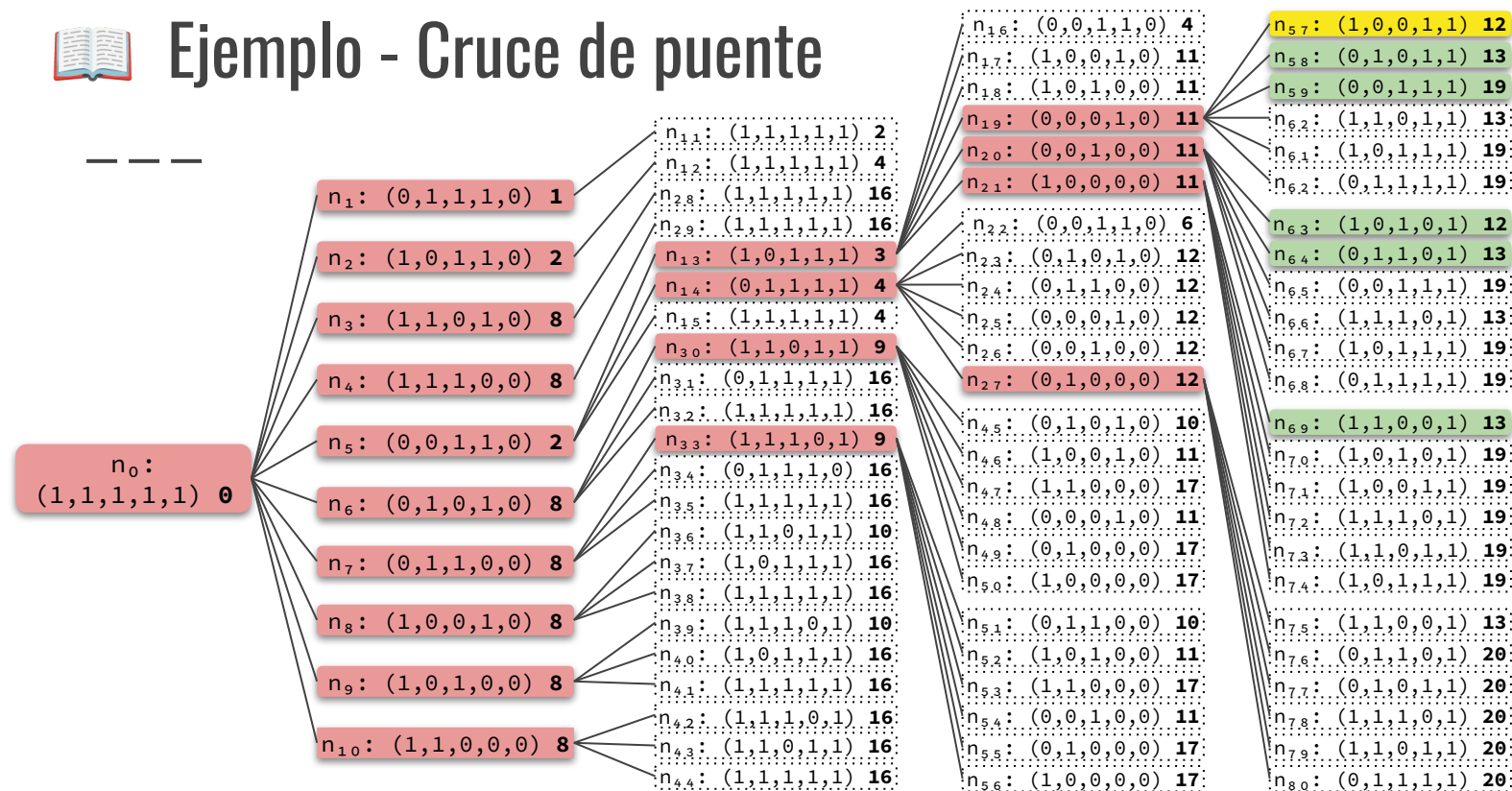


FRONTERA = $\{n_{57}, n_{58}, n_{59}, n_{63}, n_{64}, n_{69}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13\}$



Ejemplo - Cruce de puente

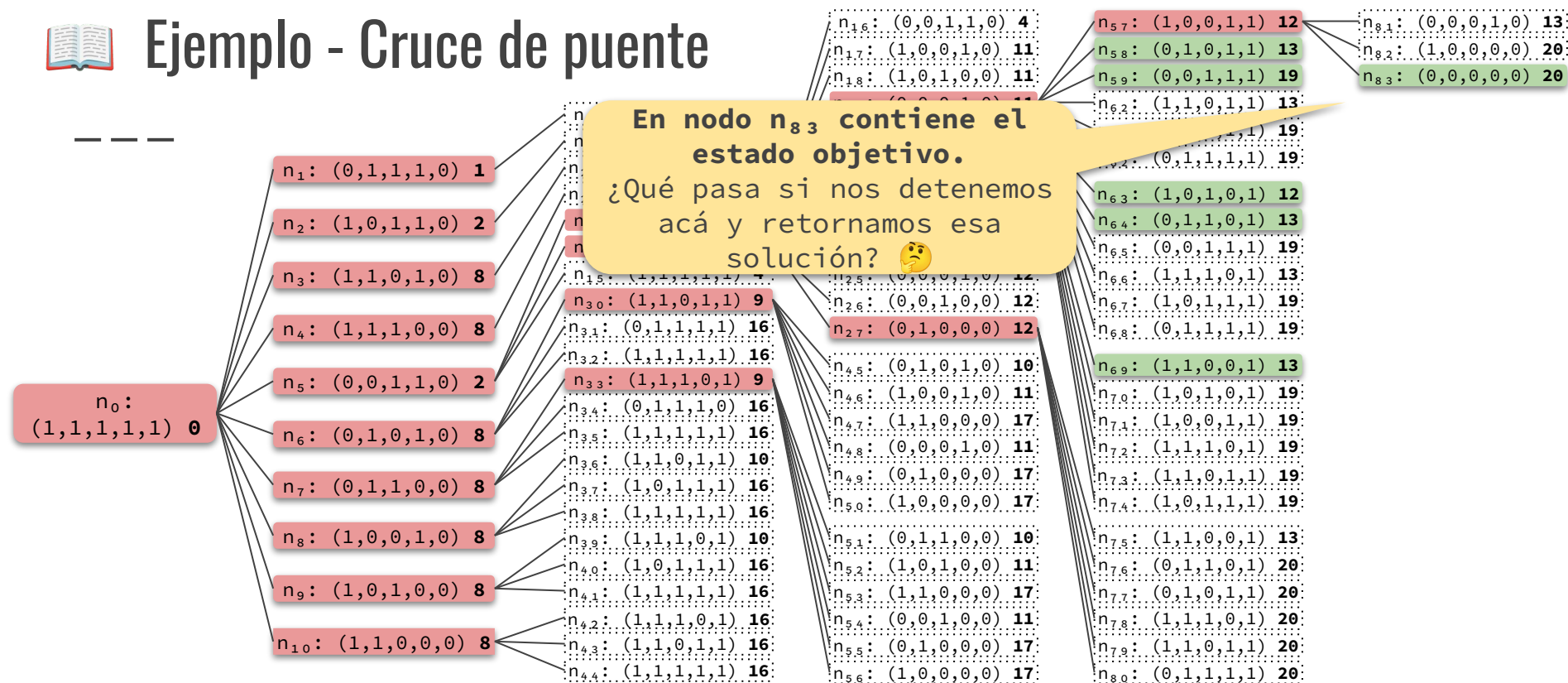


FRONTERA = $\{n_{58}, n_{59}, n_{63}, n_{64}, n_{69}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13\}$

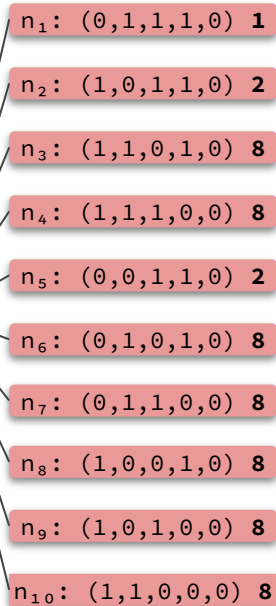


Ejemplo - Cruce de puente





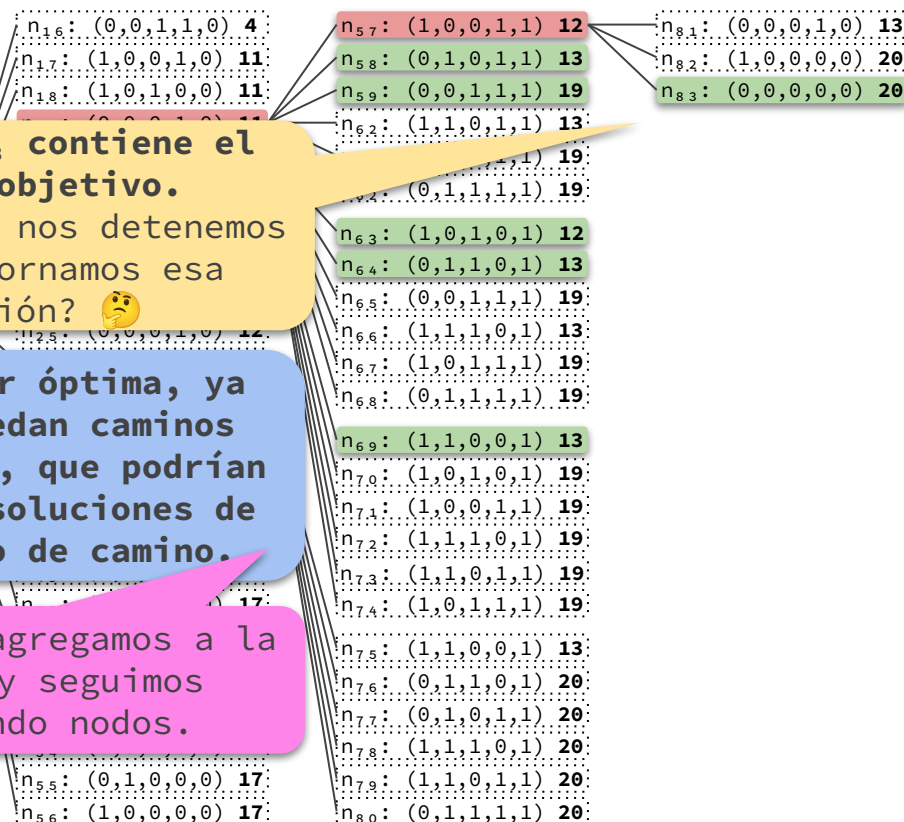
Ejemplo - Cruce de puente



En nodo n_{83} contiene el estado objetivo.
¿Qué pasa si nos detenemos acá y retornamos esa solución? 🤔

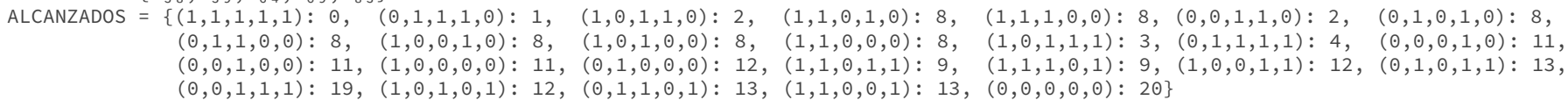
Puede no ser óptima, ya que aún quedan caminos por explorar, que podrían conducir a soluciones de menor costo de camino.

Entonces lo agregamos a la frontera y seguimos expandiendo nodos.



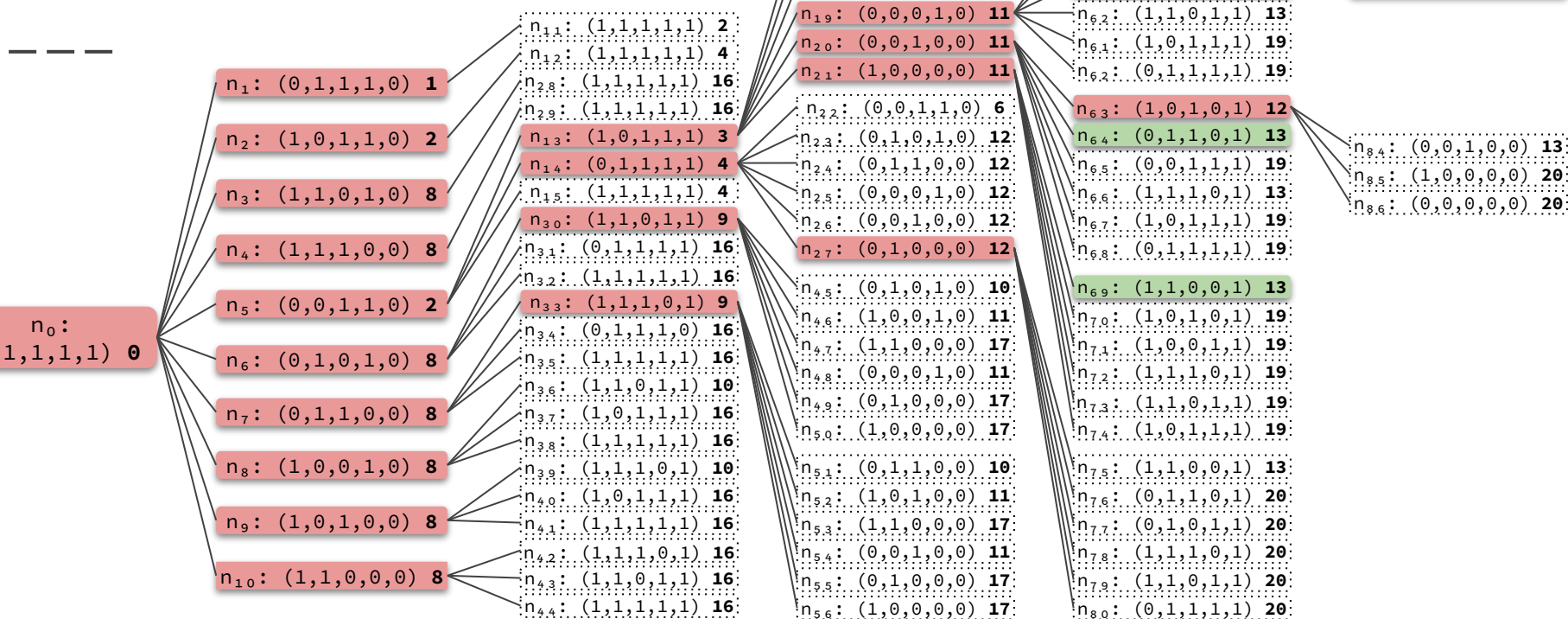
FRONTERA = $\{n_{58}, n_{59}, n_{63}, n_{64}, n_{69}, n_{83}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 12, (1,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13, (0,0,0,0,0): 20\}$





Ejemplo - Cruce de puente

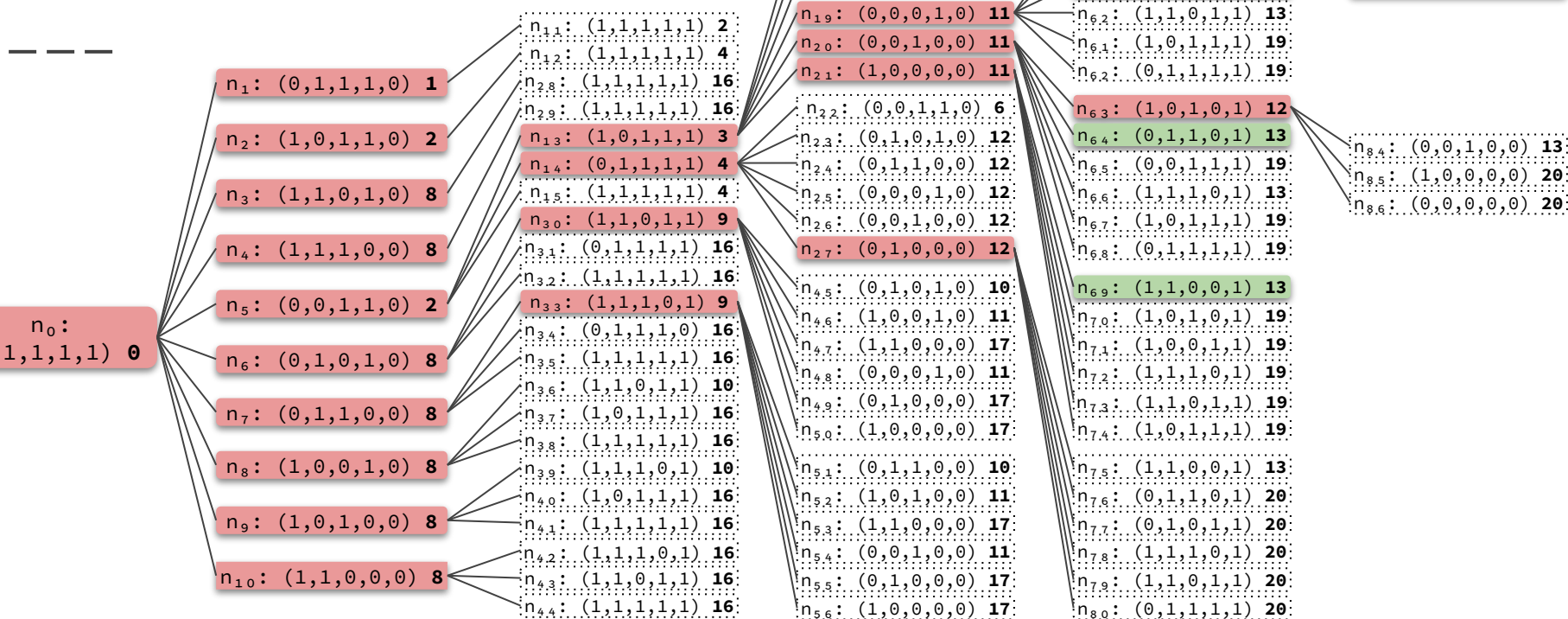


FRONTERA = $\{n_{58}, n_{59}, n_{64}, n_{69}, n_{83}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13, (0,0,0,0,0): 20\}$



Ejemplo - Cruce de puente

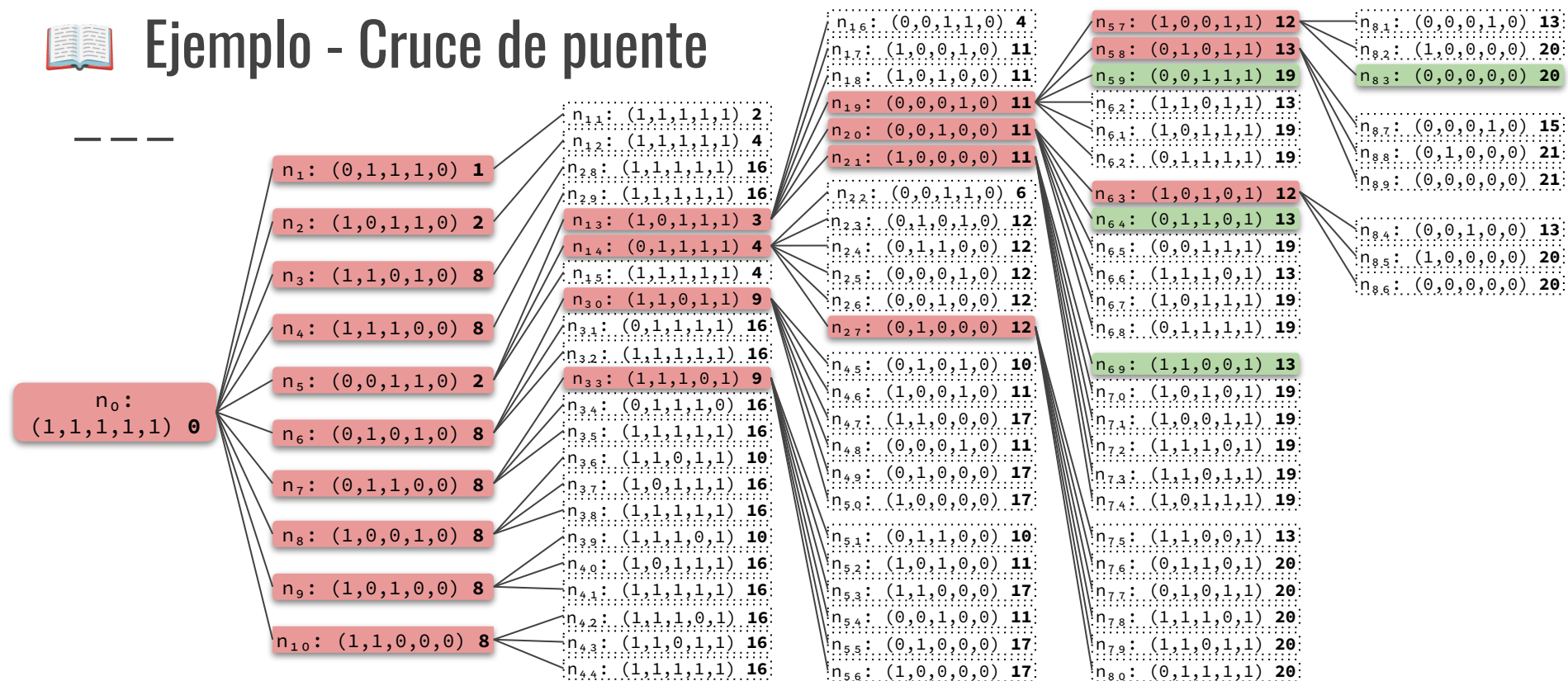


FRONTERA = $\{n_{59}, n_{64}, n_{69}, n_{83}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,0,1,0,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13, (0,0,0,0,0): 20\}$



Ejemplo - Cruce de puente

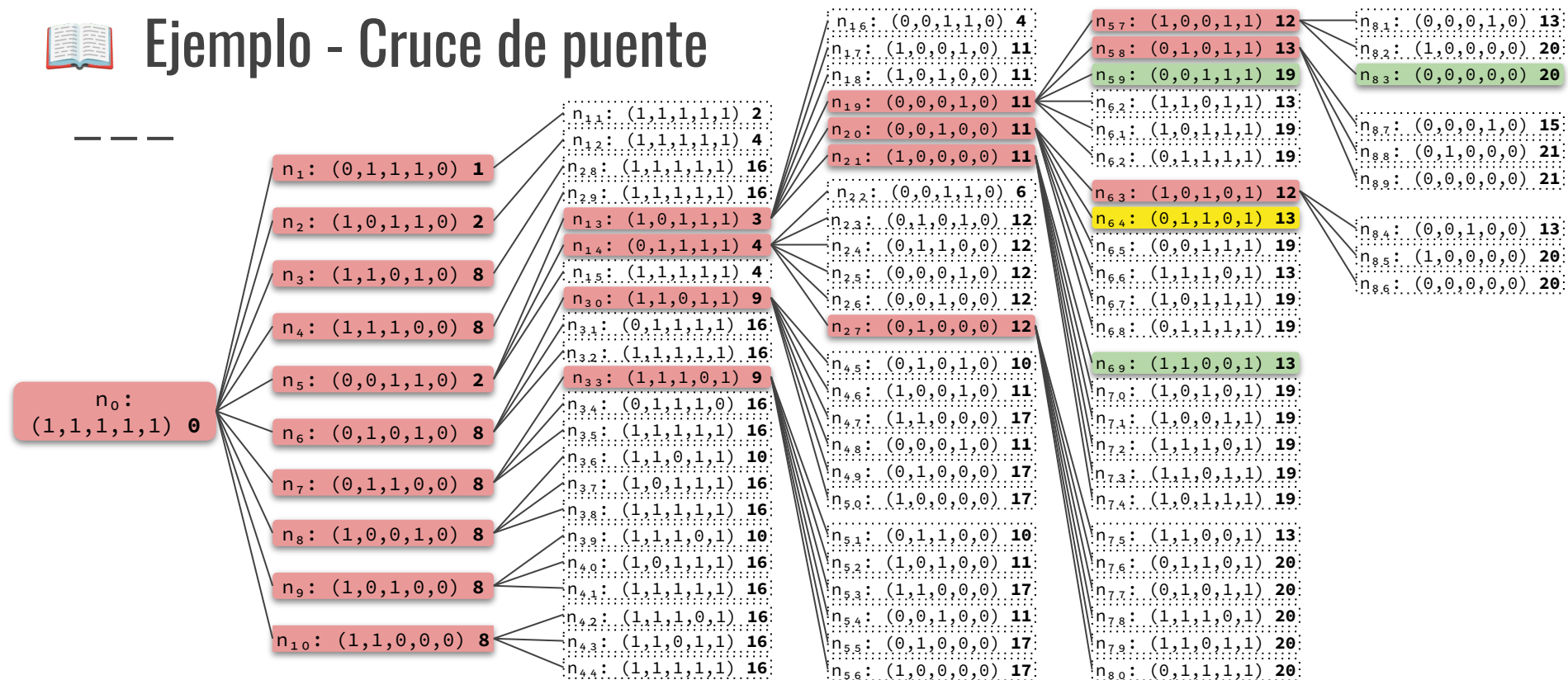


FRONTERA = $\{n_{59}, n_{64}, n_{69}, n_{83}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13, (0,0,0,0,0): 20\}$



Ejemplo - Cruce de puente

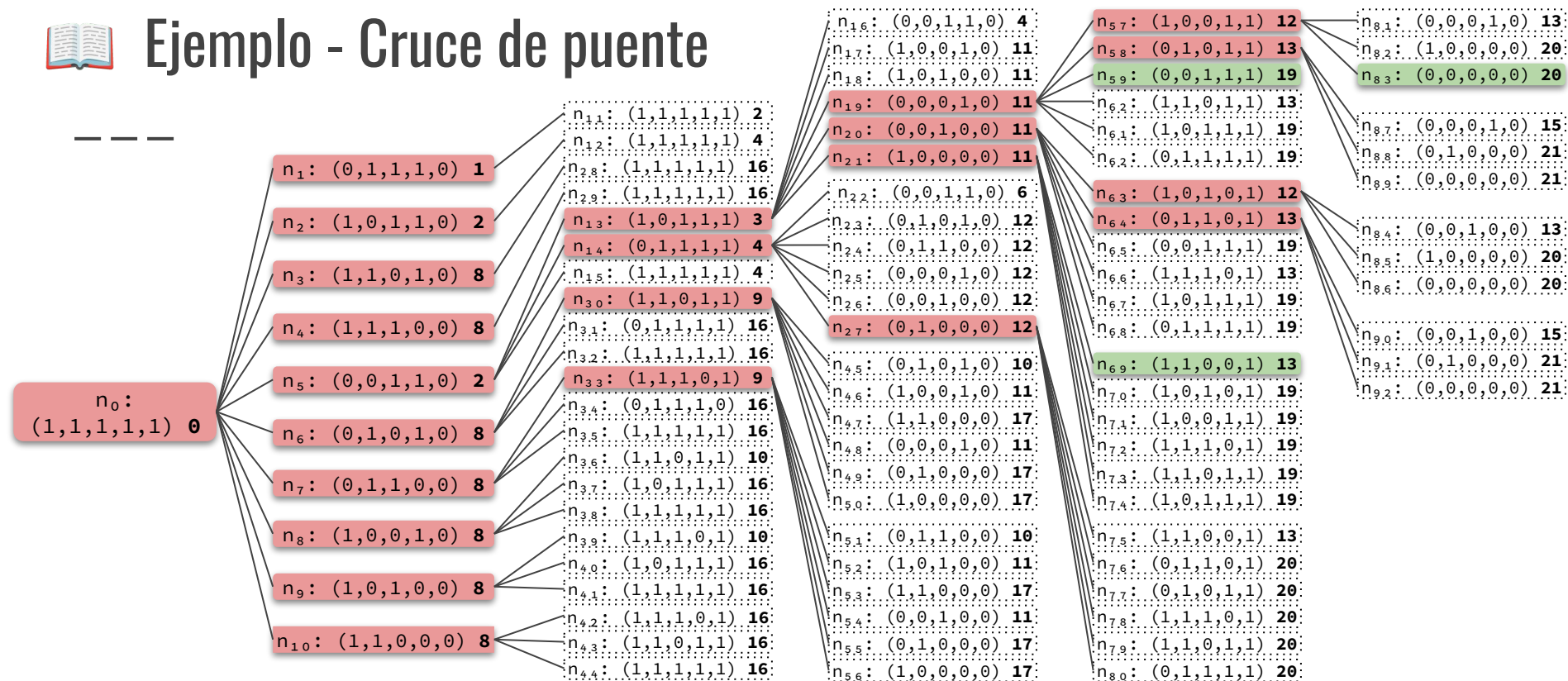


FRONTERA = $\{n_{59}, n_{69}, n_{83}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,0,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13, (0,0,0,0,0): 20\}$



Ejemplo - Cruce de puente

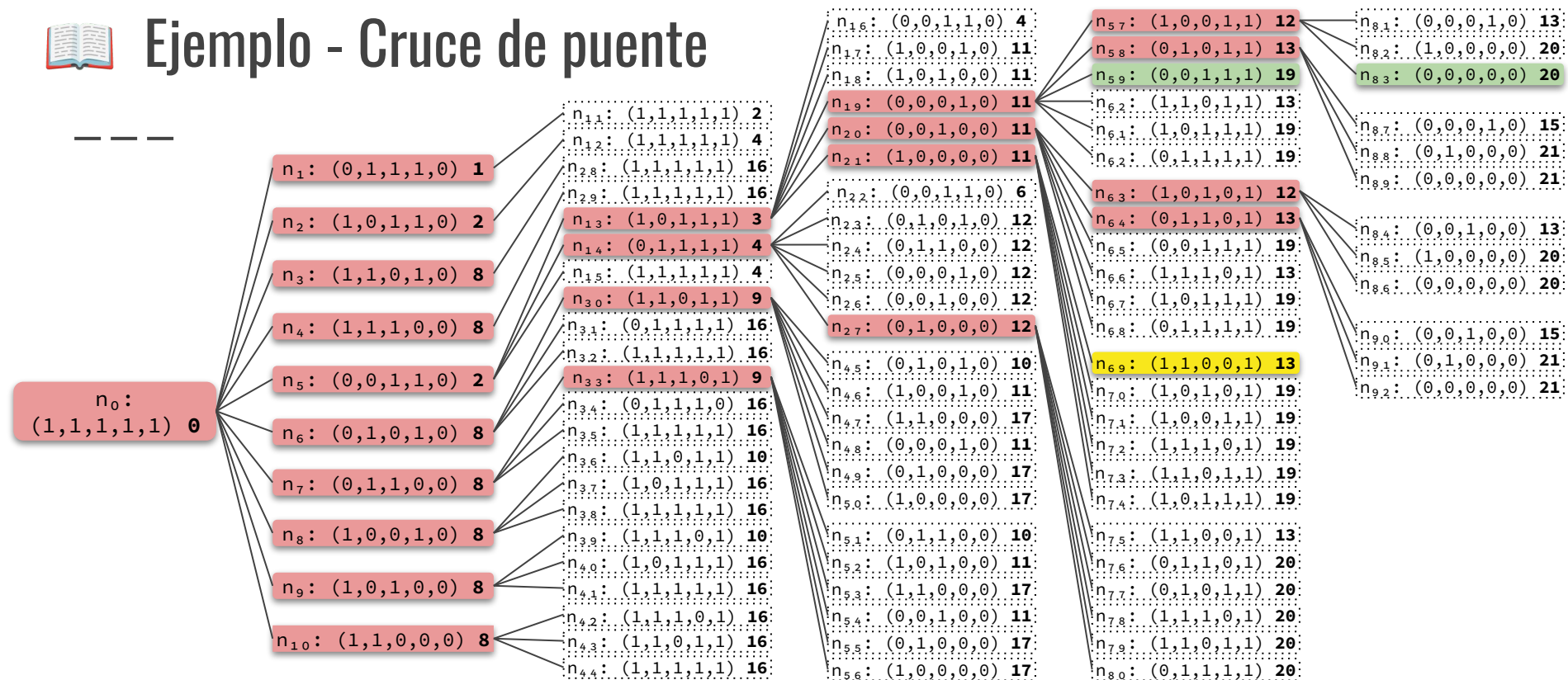


FRONTERA = $\{n_{59}, n_{69}, n_{83}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,0,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13, (0,0,0,0,0): 20\}$



Ejemplo - Cruce de puente

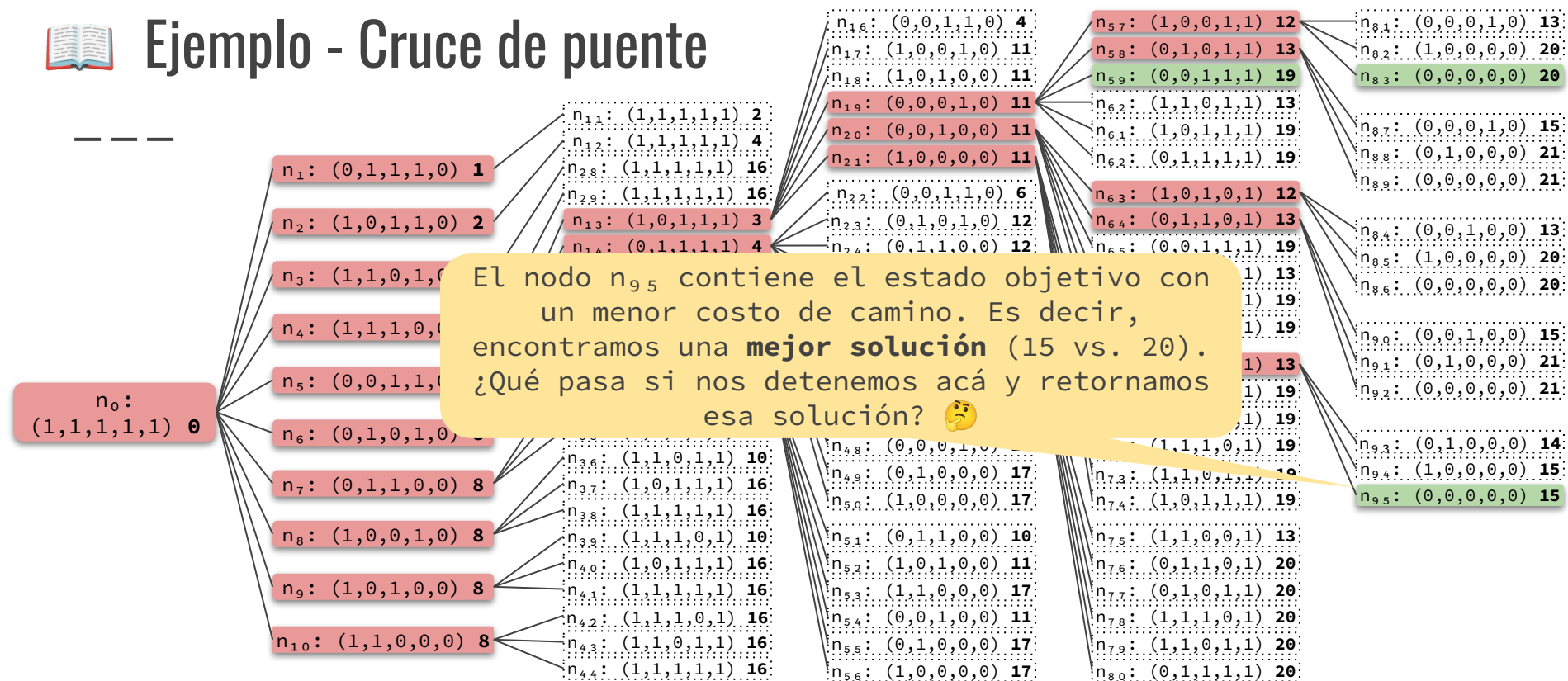


FRONTERA = $\{n_{59}, n_{83}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,1,0,0): 11, (1,0,0,0,0): 11, (0,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13, (0,0,0,0,0): 20\}$

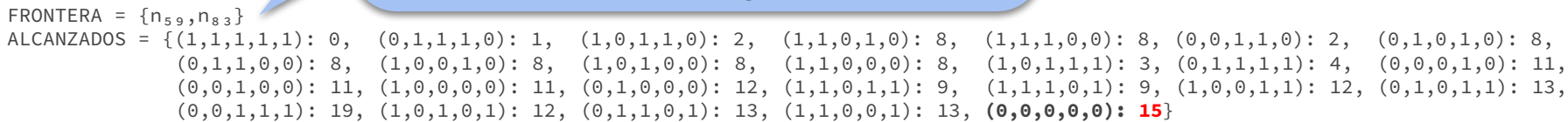


Ejemplo - Cruce de puente



FRONTERA = $\{n_{59}, n_{83}\}$

ALCANZADOS = $\{(1,1,1,1,1): 0, (0,1,1,1,0): 1, (1,0,1,1,0): 2, (1,1,0,1,0): 8, (1,1,1,0,0): 8, (0,0,1,1,0): 2, (0,1,0,1,0): 8, (0,1,1,0,0): 8, (1,0,0,1,0): 8, (1,1,0,0,0): 8, (1,0,1,1,1): 3, (0,1,1,1,1): 4, (0,0,0,1,0): 11, (0,0,0,1,0,0): 11, (1,0,0,0,0,0): 11, (0,1,1,0,0,0): 12, (1,1,0,1,1): 9, (1,1,1,0,1): 9, (1,0,0,1,1): 12, (0,1,0,1,1): 13, (0,0,1,1,1): 19, (1,0,1,0,1): 12, (0,1,1,0,1): 13, (1,1,0,0,1): 13, (0,0,0,0,0): 15\}$



Propiedades

— — —



Cuando UCS alcanza un estado (objetivo o no) por primera vez, **no garantiza** un camino de costo mínimo a ese estado.

Lo vimos en el ejemplo anterior.



Cuando UCS extrae un nodo de la frontera, entonces **garantiza** un camino de costo mínimo a ese estado.

Los demás nodos de la frontera tiene un costo de camino que es mayor o igual (de lo contrario hubiesen sido extraídos anteriormente) y no pueden conducir a caminos redundantes de menor costo (pues los costos individuales nunca son negativos).

Criterio de parada

— — —

En la estrategia UCS, es conveniente ejecutar el test objetivo ...

❑ **después de sacar un nodo de la frontera.**

~~❑ antes de agregar un nodo a la frontera.~~

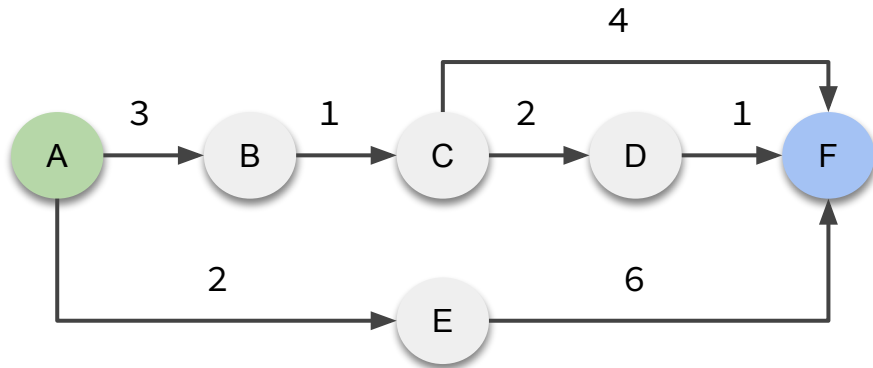
Garantiza **optimalidad**
para cualquier función de
costo individual.



Ejercicio

Sea un problema con el siguiente grafo de espacio de estados, donde A es el estado inicial y F es el estado objetivo. El costo individual de cada acción se encuentra sobre el arco correspondiente.

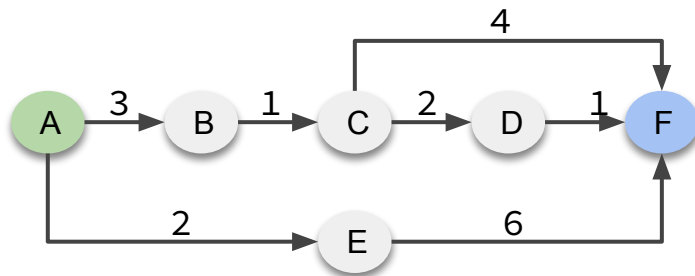
Resolverlo con el algoritmo general de búsqueda de árbol con la estrategia UCS.





Respuesta

— — —



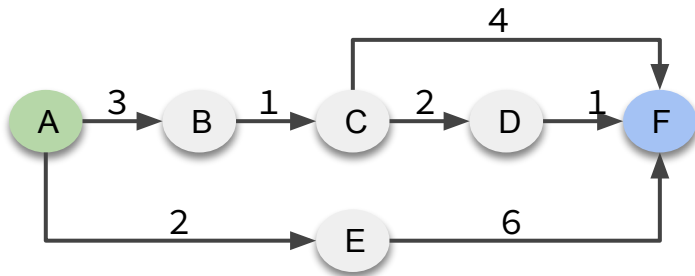
n_0
A 0

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1			
2			
3			
4			
5			
6			



Respuesta

— — —



n_0
A 0

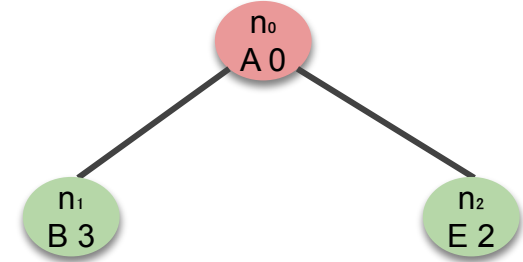
Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	
2			
3			
4			
5			
6			



Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2			
3			
4			
5			
6			

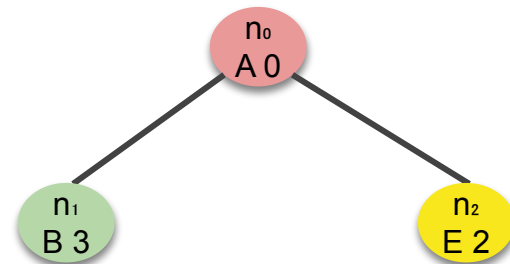




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	
3			
4			
5			
6			

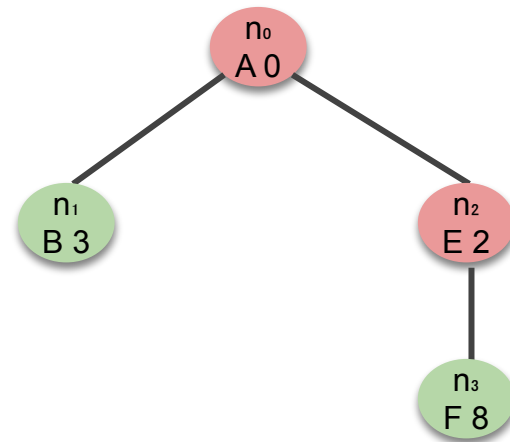




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3\}$
3			
4			
5			
6			

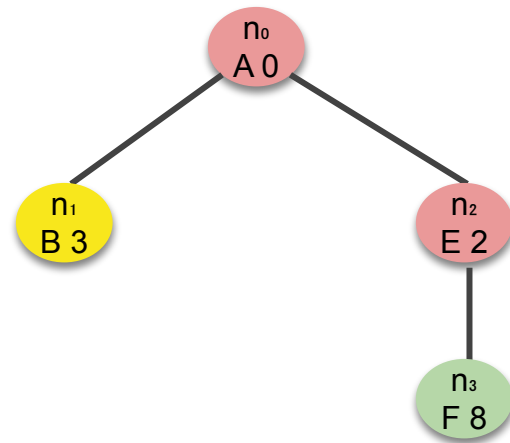




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3\}$
3	n_1	$\{n_3\}$	
4			
5			
6			

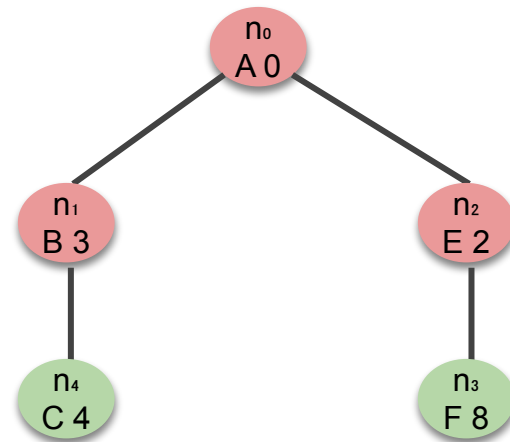




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3\}$
3	n_1	$\{n_3\}$	$\{n_3, n_4\}$
4			
5			
6			

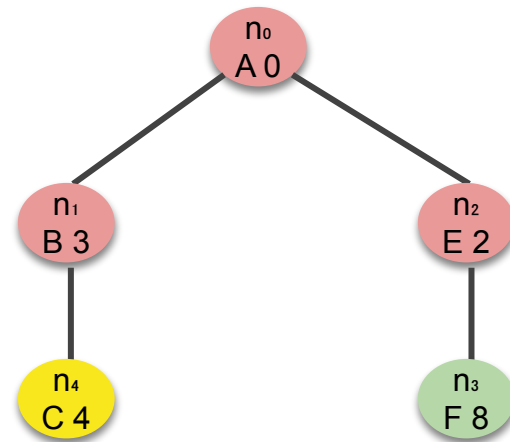




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	$\{n_0\}$	-
1	n_0	$\{\}$	$\{n_1, n_2\}$
2	n_2	$\{n_1\}$	$\{n_1, n_3\}$
3	n_1	$\{n_3\}$	$\{n_3, n_4\}$
4	n_4	$\{n_3\}$	
5			
6			

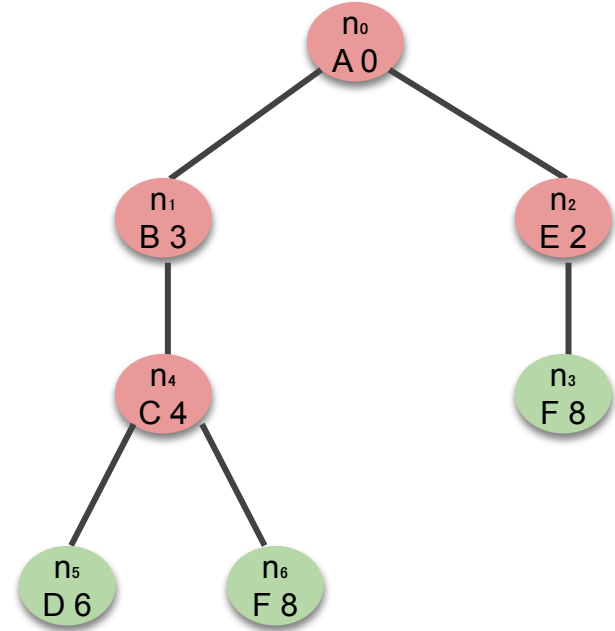




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₂	{n ₁ }	{n ₁ , n ₃ }
3	n ₁	{n ₃ }	{n ₃ , n ₄ }
4	n ₄	{n ₃ }	{n ₃ , n ₅ , n ₆ }
5			
6			

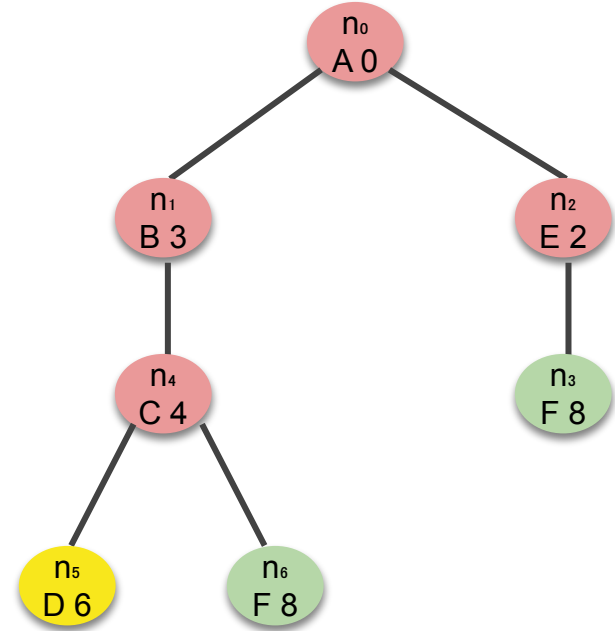




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₂	{n ₁ }	{n ₁ , n ₃ }
3	n ₁	{n ₃ }	{n ₃ , n ₄ }
4	n ₄	{n ₃ }	{n ₃ , n ₅ , n ₆ }
5	n ₅	{n ₃ , n ₆ }	
6			

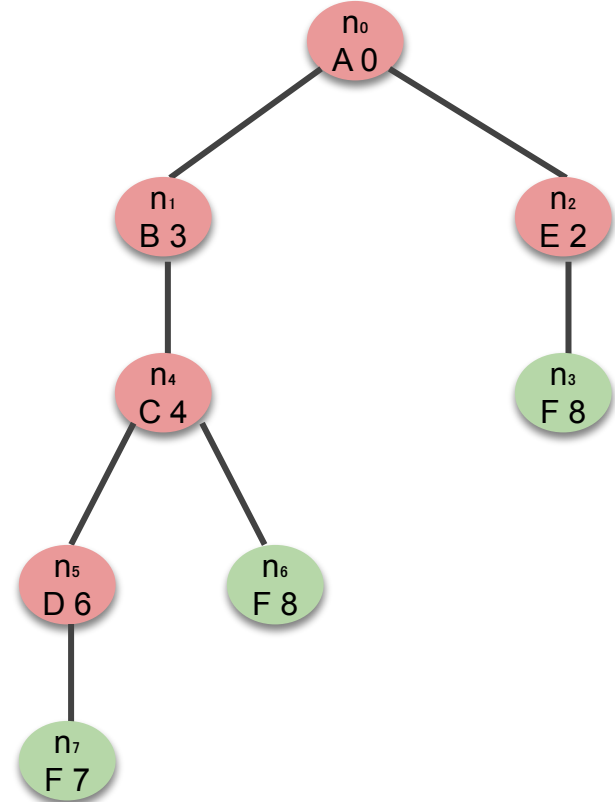




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₂	{n ₁ }	{n ₁ , n ₃ }
3	n ₁	{n ₃ }	{n ₃ , n ₄ }
4	n ₄	{n ₃ }	{n ₃ , n ₅ , n ₆ }
5	n ₅	{n ₃ , n ₆ }	{n ₃ , n ₆ , n ₇ }
6			

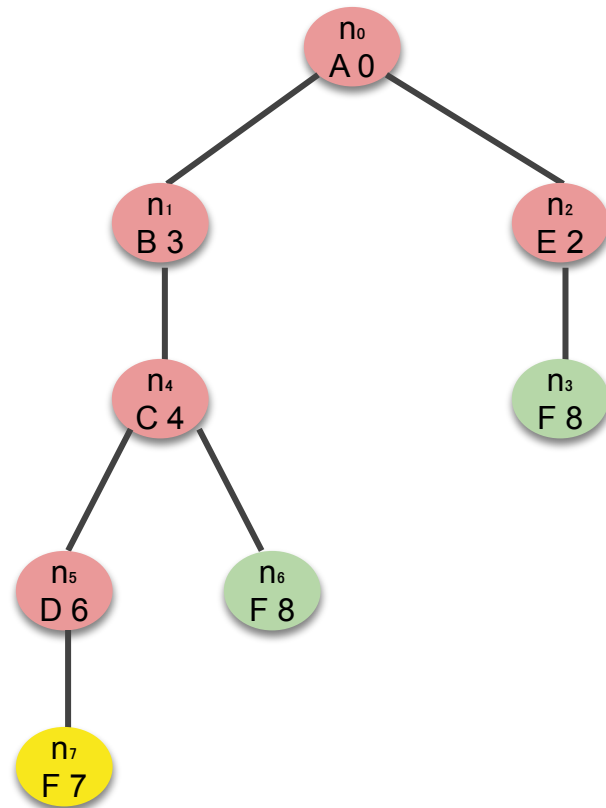




Respuesta

— — —

Nº	Nodo actual	Frontera antes de expandir	Frontera después de expandir
0	-	{n ₀ }	-
1	n ₀	{}	{n ₁ , n ₂ }
2	n ₂	{n ₁ }	{n ₁ , n ₃ }
3	n ₁	{n ₃ }	{n ₃ , n ₄ }
4	n ₄	{n ₃ }	{n ₃ , n ₅ , n ₆ }
5	n ₅	{n ₃ , n ₆ }	{n ₃ , n ₆ , n ₇ }
6	n ₇	{n ₃ , n ₇ }	¡FIN!



Implementación de UCS

- ¿Qué nodo se elige de la frontera?

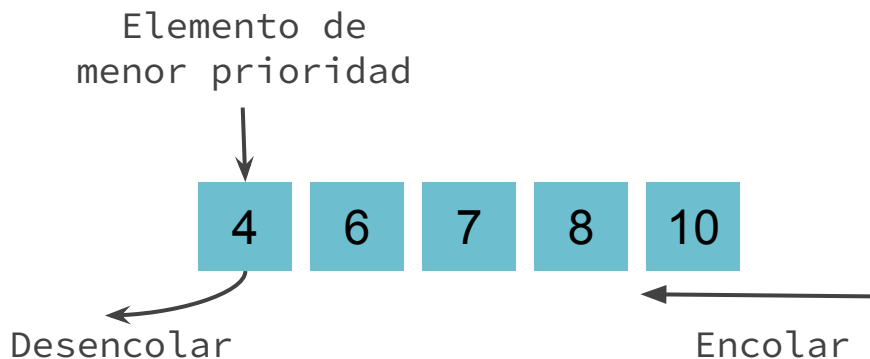
El de menor costo de camino.

- ¿Cómo lo logramos?

Si los nuevos nodos se insertan ordenados en la frontera según su costo de camino, entonces en la parte de adelante de la frontera siempre se encuentra el nodo con mínimo costo de camino.

¿Qué estructura de datos es ideal para esto? 🤔

TAD Cola de prioridad



Política: el de menor prioridad es el primero que sale.

Operaciones disponibles:

- **Encolar**: agrega un nodo en la cola con una cierta prioridad.
- **Desencolar**: elimina el nodo de la cola con la menor prioridad y lo devuelve.
- **Vacía**: determina si la cola de prioridad es vacía.



Algoritmo UCS de grafo

```
1 function GRAPH-UCS(problema) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   frontera  $\leftarrow \text{ColaPrioridad}()$ 
4   frontera.encolar( $n_0, n_0.\text{costo}$ )
5   alcanzados  $\leftarrow \{n_0.\text{estado}: n_0.\text{costo}\}$ 
6   do
7     if frontera.vacía() then return no-solución
8     n  $\leftarrow \text{frontera.desencolar}()$ 
9     if problema.test-objetivo(n.estado) then return solución(n)
10    forall a in problema.acciones(n.estado) do
11       $s' \leftarrow \text{problema.resultado}(\text{n.estado}, a)$ 
12       $c' \leftarrow \text{n.costos} + \text{problema.costos-individual}(\text{n.estado}, a)$ 
13      if  $s'$  is not in alcanzados or  $c' < \text{alcanzados}[s']$  then
14         $n' \leftarrow \text{Nodo}(s', n, a, c')$ 
15        alcanzados[ $s'$ ]  $\leftarrow c'$ 
16        frontera.encolar( $n', c'$ )
```



Algoritmo UCS de grafo

```
1 function GRAPH-UCS(problema) return solución o no-solución
2   n0 ← NODO(problema.estado-inicial, None, None, 0)
3   frontera ← ColaPrioridad()
4   frontera.encolar(n0, n0.costo)
5   alcanzados ← {n0.estado: n0.costo}
6   do
7     if frontera.vacía() then return no-solución
8     n ← frontera.desencolar()
9     if problema.test-objetivo(n.estado) then return solución(n)
10    forall a in problema.acciones(n.estado) do
11      s' ← problema.resultado(n.estado, a)
12      c' ← n.costos + problema.costos-individual(n.estado, a)
13      if s' is not in alcanzados or c' < alcanzados[s'] then
14        n' ← Nodo(s', n, a, c')
15        alcanzados[s'] ← c'
16        frontera.encolar(n', c')
```

TREE-UCS es muy similar. Hay que borrar las partes relacionadas con el diccionario de alcanzados (líneas 5, 13 y 15).

Un nodo se **descarta** únicamente cuando su estado ya había sido alcanzado con un costo de camino menor o igual.

Performance de TREE-UCS y GRAPH-UCS

— — —

- **Completitud y optimalidad.**  **Para costos individuales $> 0^*$**

* **TREE-UCS** podría no terminar si hay un camino cíclico en el espacio de estados con acciones de costo individual 0.

- **Tiempo y memoria.**

El consumo de tiempo y memoria de **TREE-UCS** sigue siendo **exponencial**, aunque el exponente depende ahora del cociente entre el costo de una solución óptima y el menor costo individual de una acción. No vamos a ver mayores detalles.

Búsqueda de profundización iterativa

Iterative-Deepening Search
(IDS)

Se introduce un **límite de profundidad** l .

En cada iteración, se aplica **DFS** sobre el **árbol de búsqueda limitado** a profundidad l .

Es decir, los nodos del árbol de búsqueda en el nivel l se tratan como si fuesen **hojas**.

Comenzando con $l = 0$, el límite se **aumenta** iterativamente en 1 hasta encontrar una solución.

— — —



Ejemplo



Nodo de la frontera.



Nodo expandido con descendientes en la frontera.

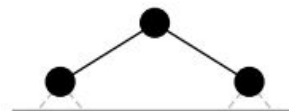
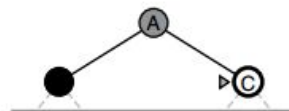
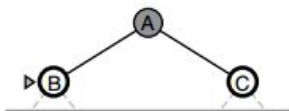
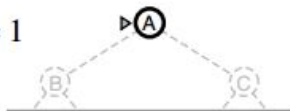


Nodo expandido sin descendientes en la frontera (se pueden liberar de la memoria).

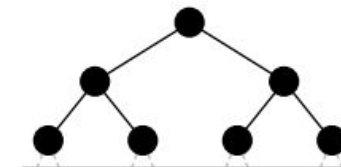
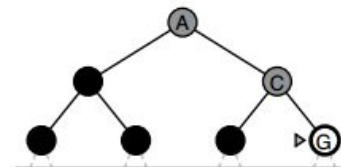
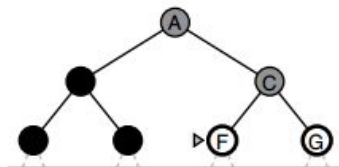
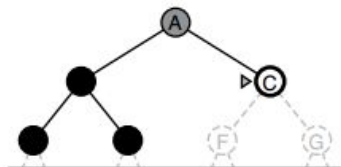
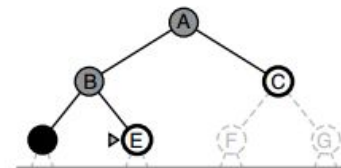
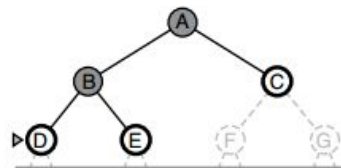
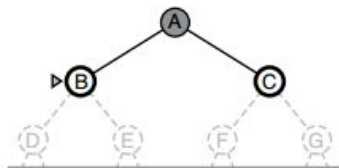
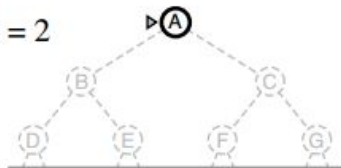
Limit = 0



Limit = 1



Limit = 2





Algoritmo IDS de árbol

— — —

¿Qué límite de profundidad elegimos? 🤔

```
1 function TREE-IDS(problema) return solución o no-solución
2   for  $l = 0$  to  $\infty$  do
3     resultado  $\leftarrow$  TREE-LIMITED-DFS(problema,  $l$ )
4     if resultado  $\neq$  seguir then resultado
```

Donde TREE-LIMITED-DFS es muy similar a TREE-DFS. Ahora la veremos...

En particular, TREE-LIMITED-DFS puede devolver una solución, un fallo o un seguir, este último indica que se alcanzó el límite de profundidad sin encontrar solución y se precisa aumentarlo.



Algoritmo IDS de árbol

```
1 function TREE-LIMITED-DFS(problema, l) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\textit{problema}.\textit{estado-inicial}, \text{None}, \text{None}, 0, \text{profundidad}=0)$ 
3   if problema.test-objetivo( $n_0$ .estado) then return solución( $n_0$ )
4   frontera  $\leftarrow$  Pila()
5   frontera.apilar( $n_0$ )
6   retorno  $\leftarrow$  no-solución
7   do
8     if frontera.vacia() then return retorno
9      $n \leftarrow$  frontera.desapilar()
10    if  $n.\text{profundidad} == l$  then retorno  $\leftarrow$  seguir
11    else
12      forall a in problema.acciones( $n$ .estado) do
13         $s' \leftarrow$  problema.resultado( $n$ .estado, a)
14         $n' \leftarrow$  Nodo( $s'$ ,  $n$ , a,  $n$ .costo+problema.costo-individual( $n$ .estado,a),
15                      profundidad= $n$ .profundidad+1)
16        if problema.test-objetivo( $s'$ ) then return solución( $n'$ )
17        frontera.apilar( $n'$ )
```



Algoritmo IDS de árbol

```
1 function TREE-LIMITED-DFS(problema, l) return solución o no-solución
2   n0 ← NODO(problema.estado-inicial, None, None, 0, profundidad=0)
3   if problema.test-objetivo(n0.estado) then return solución(n0)
4   frontera ← Pila()
5   frontera.apilar(n0)
6   retorno ← no-solución
7   do
8     if frontera.vacía() then return retorno
9     n ← frontera.desapilar()
10    if n.profundidad == l then retorno ← seguir
11    else
12      forall a in problema.acciones(n.estado) do
13        s' ← problema.resultado(n.estado, a)
14        n' ← NODO(s', n, a, n.costo+problema.costo-individual(n.estado,a),
15                  profundidad=n.profundidad+1)
16        if problema.test-objetivo(s') then return solución(n')
17        frontera.apilar(n')
```

Se precisa el atributo **profundidad** del nodo.

Si no encuentra solución retorna:
- **no-solución** si no se generó ningún nodo de profundidad **l** → **no existe solución**.
- **seguir** si se generó al menos un nodo de profundidad **l** → **se debe aumentar un nivel**.

Si alguno de los nodos extraídos se encuentra en el límite de profundidad, entonces se **descarta sin expandir** y se marca el retorno como **seguir**.

Performance de TREE-IDS

Recordemos la notación:

- b es el factor de ramificación del árbol de búsqueda, es decir, el máximo número de hijos por nodo.
- d es el menor nivel del árbol de búsqueda que contiene a una solución.

Completitud de TREE-IDS

Si existen soluciones, ¿encuentra al menos una? 🤔

Completitud de TREE-IDS

Si existen soluciones, ¿encuentra al menos una? 🤔

- La primera llamada a TREE-LIMITED-DFS genera el árbol hasta el nivel 0.
- La segunda llamada a TREE-LIMITED-DFS genera el árbol hasta el nivel 1.
- La tercera llamada a TREE-LIMITED-DFS genera el árbol hasta el nivel 2.
- ... y así sucesivamente hasta el nivel *d*.

El árbol de búsqueda se genera iterativamente por niveles.

Completitud de TREE-IDS

Si existen soluciones, ¿encuentra al menos una? 🤔

✓ **Siempre⁺ encuentra una solución** aunque no se detecten caminos cíclicos. De hecho la encuentra al llegar al límite de profundidad $l = d$.

⁺ Si el límite de profundidad es lo suficientemente grande.

Optimalidad de TREE-IDS

Si existen soluciones, ¿encuentra una óptima? 🤔

Optimalidad de TREE-IDS

Si existen soluciones, ¿encuentra una óptima? 🤔

Depende de los costos...

✓ **Para costos individuales unitarios ($= 1$), siempre encuentra una solución óptima** (por el mismo motivo que TREE-BFS).

Consumo de tiempo de TREE-IDS

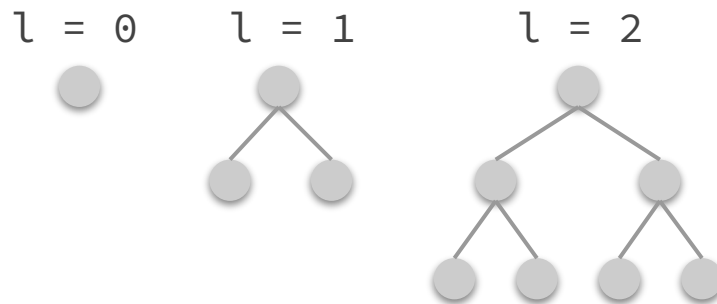
— — —

¿Cuál es el
consumo de
tiempo? 🤔

Consumo de tiempo de TREE-IDS

 **Ejemplo.** Supongamos $b = 2$ y $d = 2$.

Se generan los siguientes árboles de búsqueda.



¿Cuál es el
consumo de
tiempo? 🤔

Es decir, la raíz se genera 3 veces, cada nodo del nivel 1 se genera 2 veces y cada nodo del nivel 2 se genera a lo sumo 1 vez.

Consumo de tiempo de TREE-IDS

¿Cuál es el
consumo de
tiempo? 🤔

De manera más general:

- La raíz se genera $d+1$ veces.
- Los nodos del nivel 1 se generan d veces.
- Los nodos del nivel 2 se generan $d-1$ veces.
- ...
- Los nodos del nivel d que se generan a lo sumo 1 vez.

Luego, el número de nodos generados es a lo sumo:

$$(d+1) \cdot b^0 + d \cdot b^1 + (d-1) \cdot b^2 + \dots + 1 \cdot b^d$$

Consumo de tiempo de TREE-IDS

— — —

El consumo de tiempo de TREE-IDS puede parecer mucho mayor que el de TREE-BFS y TREE-DFS. Recordemos... El número de nodos generados por IDS es a lo sumo:

$$(\textcolor{red}{d}+1) \cdot \textcolor{teal}{b}^0 + \textcolor{red}{d} \cdot \textcolor{teal}{b}^1 + (\textcolor{red}{d}-1) \cdot \textcolor{teal}{b}^2 + \dots + 1 \cdot \textcolor{red}{b}^{\textcolor{red}{d}}$$

y por BFS y DFS es a lo sumo:

$$\textcolor{teal}{b}^0 + \textcolor{teal}{b}^1 + \textcolor{teal}{b}^2 + \dots + \textcolor{red}{b}^{\textcolor{red}{d}}$$

 **Ejemplo.** Para $\textcolor{teal}{b} = 10$ y $\textcolor{red}{d} = 5$:

$$6 + 50 + 400 + 3000 + 20000 + 100000 = 123456 \text{ (IDS)}$$

$$1 + 10 + 100 + 1000 + 10000 + 100000 = 111111 \text{ (DFS)}$$

La diferencia no es significativa, pues la mayoría de los nodos del árbol de búsqueda se encuentran en el último nivel, y es menos significativa a mayor factor de ramificación.

Consumo de tiempo de TREE-IDS

— — —

¿Cuál es el consumo de memoria? 🤔

Consumo de tiempo de TREE-IDS

¿Cuál es el consumo de memoria? 🤔

El consumo de memoria de TREE-IDS depende exclusivamente del consumo de memoria de TREE-LIMITED-DFS.

Dado que TREE-LIMITED-DFS es muy similar a TREE-DFS, ambos algoritmos tienen el mismo consumo de memoria, es decir, es:

$b.d$

Performance de TREE-IDS

- **Completitud.** ✓
- **Optimalidad.** ✓ Si las acciones tienen costo individual unitario.
✗ En otros casos.
- **Tiempo.** Se generan a lo sumo b^{d+1} nodos.
- **Memoria.** Se mantienen a lo sumo $b \cdot d$ nodos en memoria.

Combina las ventajas de **TREE-BFS** (completitud y optimalidad) y **TREE-DFS** (memoria). Es el algoritmo de búsqueda de árbol preferido en grandes espacios de búsqueda y con pocos caminos redundantes.



Algoritmo IDS de grafo

— — —

- La adaptación de **TREE-IDS** a **GRAPH-IDS** no es muy popular.
- Su consumo de memoria **empeora**, deja de ser lineal en la altura del árbol, y pasa a ser exponencial en la altura del árbol (por el almacenamiento de los estados expandidos), tal como pasaba al adaptar **TREE-DFS** a **GRAPH-DFS**.
- **No tiene ventajas sobre **GRAPH-BFS**, y se sigue prefiriendo este último como algoritmo de búsqueda de grafo.**



Próximamente

— — —

Las estrategias de búsqueda que vimos hasta ahora toman sus decisiones basadas únicamente en la formulación del problema.

Veremos **estrategias de búsqueda informadas**, cuyas decisiones están guiadas por una función **heurística** que estima el costo de una solución desde un estado en particular.